

CSE 4000: Thesis/ Project

**ROAD CONDITION-AWARE TRAFFIC ROUTE
OPTIMIZATION FOR
MEGACITY DHAKA USING ANT COLONY
OPTIMIZATION**

By

Sourav Debnath

Roll: 2007109



**Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
July, 2026**

Road Condition-Aware Traffic Route Optimization for Megacity Dhaka Using Ant Colony Optimization

By

Sourav Debnath

Roll: 2007109

A thesis submitted in partial fulfillment of the requirements for the degree of
“Bachelor of Science in Computer Science & Engineering”

Supervisor:

Dr. Muhammad Aminul Haque Akhand

Professor

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Signature

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

July, 2026

Acknowledgment

All praise is due to the Almighty for the strength and opportunity to complete this thesis. I am deeply grateful to my supervisor, Dr. Muhammad Aminul Haque Akhand, Professor, Department of Computer Science and Engineering, KUET, for his continuous guidance and encouragement in formulating the problem, refining the cost model, and interpreting the results. I thank the faculty and staff of the department for the resources that made this work possible, and the authors of the base and related studies cited throughout this report, whose work shaped this framework. Finally, I thank my parents, family, and friends for their unwavering support throughout this undergraduate thesis.

Author

Abstract

Dhaka, one of the world’s most densely populated megacities, suffers severe traffic congestion, yet conventional shortest-path routing considers only distance and ignores congestion, vehicle-class restrictions, and network-wide traffic distribution. This thesis proposes a congestion-aware, capacity-constrained route optimization framework for the central Dhaka road network using an improved Ant Colony Optimization (ACO) approach. The network is modeled as a directed graph of 1,168 nodes and 3,473 links attributed with road class, lane count, and length; five vehicle categories—motorcycle, CNG auto-rickshaw, private car, bus, and rickshaw/bicycle—are represented through Passenger Car Unit weighting and a road-class access matrix. A multi-factor road cost combining distance, lane-based capacity, access permission, and congestion level serves as the ACO heuristic. Two strategies are implemented in C++ and compared under identical parameters: Independent ACO (baseline) and the proposed Priority-Discouragement ACO, which processes 500 origin–destination queries in priority order and discourages pheromone on heavily loaded links. For fair comparison with the state of the art, five related methods were re-implemented on the same complete Dhaka dataset and evaluated through a common evaluation layer that reports identical metrics for every method while provably leaving all native outputs unchanged. Results show the proposed method reduces total cost by about 8.3%, average congestion by 26.3%, and improves load balance by 12.0% (29.1% by Jain’s index), routing exactly the same 351 of 500 feasible queries as the baseline in 13.3 seconds, and it is the only compared method that jointly optimizes hundreds of simultaneous queries at network scale.

Contents

Acknowledgment	iii
Abstract	iv
List of Tables	x
List of Figures	xiii
List of Abbreviations	xiv
List of Symbols	xv
I Introduction	1
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 Research Gap	3
1.4 Research Aim	3
1.5 Research Objectives	3
1.6 Significance of the Study	4
1.7 Scope of the Study	4
1.8 Limitations of the Study	5
1.9 Thesis Organization	5
II Literature Review	7
2.1 Background of the Problem	7
2.2 Significance of Computational Intelligence in Traffic Route Optimization	8
2.3 Benchmark Dataset and Procedure of Data Collection	8
2.4 Review of Previous Studies	9
2.4.1 Shortest-Path and Heuristic Methods	9
2.4.2 Improved ACO Methods	9
2.4.3 Combined and Hybrid Methods	10
2.4.4 Dhaka City Road-Network-Based Studies	11
2.5 Summary of Existing Methods	11
2.6 Critical Analysis of Previous Work	12
2.7 Identified Research Gap	13

2.8	Conceptual Framework and Proposed Research Direction	13
2.9	Chapter Summary	14
III	Research Methodology	15
3.1	Research Framework	15
3.1.1	Problem Formulation	15
3.2	Preliminary: Ant Colony Optimization and Its Adaptation	16
3.2.1	Classical Ant Colony Optimization	16
3.2.2	Adaptation for Congestion-Aware, Multi-Query Routing	17
3.3	Data Collection and Preparation	18
3.4	Data Preprocessing and Feature Generation	23
3.5	Congestion-Aware Cost Model	25
3.6	Improved Ant Colony Optimization Framework	26
3.6.1	Method 1: Independent ACO-Based Route Optimization	28
3.6.2	Method 2: Priority-Discouragement ACO-Based Route Opti- mization	29
3.6.3	Flowchart of the Proposed Method	31
3.7	Experimental Design and Validation Strategy	31
3.7.1	Matched-Conditions Comparison Design	32
3.7.2	Manual Path Evaluation	32
3.7.3	Common Evaluation Layer for Cross-Method Comparison	32
3.8	Evaluation Metrics	33
3.9	Novelty and Significance of the Proposed Method	34
IV	Experimental Studies	35
4.1	Experimental Environment	35
4.1.1	Project Structure and Source Code Modules	35
4.1.2	Resource Budget	35
4.2	Parameter Settings	35
4.3	Baseline and Comparative Methods	36
4.4	Procedure of Experimentation	36
4.5	Experimental Results	39
4.5.1	Result of Independent ACO	39
4.5.2	Result of Priority Discouragement ACO	40
4.5.3	Common-Layer Results for the Proposed Methods	40
4.6	Experimental Analysis	41
4.6.1	Route Consolidation and Query-Success Analysis	41
4.6.2	Query Success by Vehicle Class and Priority	41
4.6.3	Manual Route Modification Case Study	42
4.6.4	Contribution of the Coordination Mechanism	43

4.6.5	Runtime and Scalability Analysis	43
4.7	Comparative Analysis	43
4.7.1	Cross-Method Comparison Through the Common Evaluation Layer	44
4.7.2	Effect of Dataset Completeness	45
4.7.3	Within-Family Analyses of the Re-implemented Methods	46
4.8	Discussion of Findings	46
4.9	Chapter Summary	48
V	Engineering, Ethical, and Professional Considerations	53
5.1	Introduction	53
5.2	Intellectual Property Considerations	53
5.3	Ethical Considerations	54
5.4	Data Privacy and Security Considerations	54
5.5	Safety Considerations	54
5.6	Legal Considerations	55
5.7	Societal, Health, and Cultural Impact	55
5.8	Environmental and Sustainability Considerations	55
5.9	Summary	56
VI	Complex Engineering Problems and Activities	57
6.1	Introduction	57
6.2	Complex Engineering Problems Associated with the Project	57
6.3	Complex Engineering Activities Associated with the Project	59
6.4	Mapping of the Proposed Work with Complex Engineering Attributes .	61
6.5	Summary	62
VII	Conclusion and Future Work	64
7.1	Summary of the Study	64
7.2	Major Findings	64
7.3	Limitations of the Study	65
7.4	Future Recommendation	66
7.5	Closing Statement	67
	References	68
A	Representative Source Code Excerpts	72
A.1	Ant Path Construction (IndependentACO.cpp)	72
A.2	Traffic Flow Generation (FlowGenerator.cpp)	72
A.3	Cost Model Module (CostModel.cpp)	73
A.4	Priority Score and Discouragement Mechanism	73
A.5	Metrics Calculation Module (Metrics.cpp)	74

A.6	Common Evaluation Layer Output	74
A.7	Sample Dataset Records	74
A.8	Reproducing the Experiments	75

List of Tables

II.1	Summary of directly related works and their shortcomings relative to this thesis.	12
III.1	Terms of the classical ACO transition probability, Equation III.1.	17
III.2	Terms of the classical pheromone-update rule, Equation III.2.	17
III.3	Notation used throughout the methodology (classical ACO symbols and this thesis's extensions).	18
III.4	Summary of input datasets used for the Dhaka road-network route optimization experiments.	21
III.5	Vehicle classes, Passenger Car Unit (PCU) weights, and average vehicle length used in this thesis.	22
III.6	Vehicle-class access permission by road type. $\checkmark$ = permitted ($A = 1$), ? = limited ($A = 0.5$), $\times$ = prohibited ($A = 0$); values are used directly in the cost function of Section 3.5.	22
III.7	Distribution of the 500 origin–destination queries by vehicle class.	22
III.8	Key configuration parameters used for the reported experiments.	23
III.9	Reduction from the raw collected records to the cleaned network loaded by the solver.	24
IV.1	Implementation environment used for developing and running the route optimization system.	36
IV.2	Source-code modules and the methodology section each implements.	37
IV.3	Approximate resource budget for the thesis project.	37
IV.4	The six implementation projects included in the common cross-method comparison.	38
IV.5	Output files generated by the route optimization system.	39
IV.6	Excerpt of <code>method1_paths.csv</code> for query 0 (node 1155 to node 355, private car).	39
IV.7	Common-layer measurements for the two proposed methods on the 500-query complete Dhaka dataset.	40
IV.8	Query success rate by vehicle class over the 500-query set (identical for both proposed methods).	41

IV.9	Manual path-evaluation case study for query 0 (37-vehicle private-car trip, node 1155 to node 355).	42
IV.10	Solve-time scaling of the proposed methods from the sparse extract to the complete Dhaka dataset (500 queries in both cases).	43
IV.11	Comparison of Independent ACO and Priority-Discouragement ACO on the 500-query complete Dhaka dataset (full contents of <code>metrics.csv</code>).	44
IV.12	Cross-method comparison of all eleven method rows through the common evaluation layer (complete-dataset run of 4 July 2026). Cost and congestion units differ per paper and must not be compared across projects; capabilities, success rates, runtimes, and fairness indices are comparable.	47
IV.13	Proposed methods on the earlier sparse extract versus the complete Dhaka dataset. Because the two datasets have different link sets, the meaningful reading is the direction of change and the gap between the two methods, not the raw magnitudes.	50
VI.1	Mapping of the thesis to Complex Engineering Problem (CEP) attributes.	57
VI.2	Mapping of the thesis to Complex Engineering Activity (CEA) attributes.	60
VI.3	Consolidated mapping of major system components to CEP/CEA attributes.	63
A.1	Aggregated common-layer comparison (<code>final_common_comparison.csv</code> , complete-dataset run of 4 July 2026). “L.B.” is the Jain fairness index; N/A entries are honest by-design gaps documented in each row’s notes field.	75

List of Figures

I.1	A snapshot of severe traffic congestion on the Farmgate–Karwan Bazar corridor in Dhaka, illustrating the everyday scale of the routing problem addressed in this thesis.	2
I.2	Gantt chart showing the week-by-week project plan across the first and second terms of the thesis.	6
III.1	Overall architecture of the proposed congestion-aware ACO route optimization system, organized as five layers: (1) user OD query, (2) input data layer, (3) processing layer, (4) ACO routing (Independent ACO and Priority-Discouragement ACO), and (5) output and evaluation.	16
III.2	Central Dhaka study area used for road-network dataset preparation, with a 10 km-radius core region (orange circle) inside a 17×28 km bounding region (blue rectangle).	19
III.3	Manual lane counting with Google Street View. The pegman (the “yellow man”) is dropped onto a road segment – here Kazi Nazrul Islam Avenue – and the lane count is read off by observing how many vehicles can move side by side along the carriageway, cross-checked against the visible lane-divider markings.	20
III.4	Illustration of arterial, collector, and local road categories used in the access matrix.	20
III.5	Five vehicle categories considered in the PCU-based traffic model. . . .	21
III.6	Composition of the cleaned Dhaka link set: road-type distribution (left) and lane-count distribution (right). Arterial corridors dominate, and most links are one- or two-lane segments.	24
III.7	Components of the road-factor / congestion-aware cost model: node-to-node distance d_{ij} , lane count, road capacity, and the flow-input/flow-output/flow-on-road quantities used in Equations III.5–III.7.	26
III.8	Illustration of Passenger Car Unit (PCU) conversion for heterogeneous vehicle types.	26

III.9	Independent ACO workflow (Method 1): for each query, the network model and candidate paths are built and evaluated independently, vehicles are allocated to the best paths, and assignments are merged before metrics are computed. Route selection relies solely on the pheromone and heuristic values, with no cross-query interaction.	29
III.10	Priority-Discouragement ACO workflow (Method 2, proposed): queries are sorted by importance $I(q)$ and a shared ACO state is initialized once; each query in turn generates and evaluates feasible candidate paths, updates the traffic state, and discourages the links it used, before the next query is processed in priority order.	30
III.11	Flowchart of the proposed congestion-aware ACO methodology, annotated with the road factor RF_{ij} , heuristic η_{ij} , transition rule P_{ij}^k , and pheromone update $\tau \leftarrow (1 - \rho)\tau + \Delta\tau$ used in this thesis.	31
IV.1	Query success rate by vehicle class (identical for both proposed methods). The two access-restricted classes – bus (no local roads) and rickshaw/bicycle (no arterial roads) – account for almost all of the failures. .	42
IV.2	Native-metric comparison of Independent ACO and Priority-Discouragement ACO on the complete Dhaka dataset: total cost, average congestion, and coefficient-of-variation load balance.	44
IV.3	Relative change of Priority-Discouragement ACO over the Independent ACO baseline on the complete Dhaka dataset (negative is favourable for cost and congestion; positive is favourable for the two load-balance measures). Runtime rises by a factor of 2.04.	45
IV.4	Log-scale comparison of total cost, average congestion, and load balance for Independent ACO and Priority-Discouragement ACO on the complete Dhaka dataset, with the exact values of Table IV.11 annotated on each point.	46
IV.5	Success-rate comparison of the method rows reported in Table IV.12 (routing and network-design rows; signal-only rows omitted as success rate is N/A by design).	48
IV.6	Runtime comparison (log scale) of the eleven method rows reported in Table IV.12. Zhang rows share one pipeline time; the base paper’s runtime includes PSO parameter tuning.	49
IV.7	Jain fairness index of the rows where per-link load balance is defined. Definitions of the underlying load vector differ per family (see table footnotes), so within-family comparison is the intended reading.	49

IV.8	Node and directed-link coverage of each project’s cleaned working network, compared with the raw collection (dashed line). Each implementation applies its own cleaning rule, so the retained subgraph differs per project.	50
IV.9	Effect of dataset completeness on the proposed Priority-Discouragement ACO: network size, served queries, and success rate on the sparse extract versus the complete Dhaka dataset.	50
IV.10	Average congestion and Jain fairness index for both proposed methods on the sparse extract versus the complete dataset. The link sets differ between datasets, so the comparison documents the dataset effect rather than a like-for-like magnitude change.	51
IV.11	Within-family comparison of the four Zhang <i>et al.</i> (2023) methods on the Dhaka accident scenario: native average delay and average completed-trip travel time. Lower is better; the combined method D is best.	51
IV.12	Cheng (2023) improved-ACO re-implementation: comprehensive path cost and mean path congestion for the base (midday) and peak scenarios on the same single origin–destination query.	52

List of Abbreviations

ACO	Ant Colony Optimization
AC	Average Congestion
AHP	Analytic Hierarchy Process
BLN	Bicycle Lane Network
BPR	Bureau of Public Roads (travel-time function)
CEA	Complex Engineering Activity
CEP	Complex Engineering Problem
CI	Computational Intelligence
CNG	Compressed Natural Gas (three-wheeler auto-rickshaw)
CR	Congestion Rate
CSE	Computer Science and Engineering
CSV	Comma-Separated Values
CV	Coefficient of Variation
GIS	Geographic Information System
IEEE	Institute of Electrical and Electronics Engineers
KUET	Khulna University of Engineering & Technology
LB	Load Balance
OD	Origin–Destination
PCU	Passenger Car Unit
PSO	Particle Swarm Optimization
RF	Road Factor
TC	Total Cost
TSP	Travelling Salesman Problem

List of Symbols

$G = (V, E)$	Directed road-network graph with node set V and link set E
e	A directed road link (edge)
d_e	Length of link e (meters)
λ_e	Lane count of link e
r_e	Road-type classification of link e (arterial, collector, or local)
q	Vehicle class index ($Q = \{0, \dots, 4\}$: motorcycle, CNG, car, bus, rickshaw)
pcu_q	Passenger Car Unit weight of vehicle class q
ℓ_q	Average physical length of vehicle class q (meters)
$A[r][q]$	Access-permission matrix (road type r , vehicle class q)
$\text{traffic}[e][q]$	Current vehicle count of class q on link e
L_e	Road load of link e (PCU)
C_e	Congestion coefficient of link e
cap_e	PCU throughput capacity of link e
$\text{cost}_e(q)$	Modified road cost of link e for vehicle class q
$J(P, q)$	Path cost of path P for vehicle class q
$\text{cap}(P, q)$	Path capacity of path P for vehicle class q
τ_e	Pheromone intensity on link e
$\eta_e(q)$	Heuristic desirability of link e for vehicle class q
α, β	ACO pheromone and heuristic weighting exponents
ρ	Pheromone evaporation rate
τ_0	Initial pheromone value
$I(q)$	Priority-importance score of query q
w_p, w_d	Priority and distance weighting coefficients in $I(q)$
$d^*(q)$	Shortest topological (distance-only) path length for query q
ϕ_e	Usage fraction of link e by a query's assigned paths
θ	Pheromone discouragement strength
TC	Total cost metric
AC	Average congestion metric
LB	Load balance metric
u_e	Utilization of link e (L_e/cap_e)
$\bar{u}, \sigma_u$	Mean and standard deviation of link utilization
$\mathcal{J}$	Jain's fairness index over per-link loads (common evaluation layer)
ε	Small numerical-stability constant (10^{-9})

CHAPTER I

Introduction

Megacity Dhaka's severe and worsening traffic congestion exposes the limits of conventional, congestion-blind shortest-path routing, motivating the network-level, congestion-aware Ant Colony Optimization framework that this thesis develops, evaluates, and bounds in scope below.

1.1 Background of the Study

Urban route optimization is essential for Dhaka because conventional shortest-path selection ignores congestion, road capacity, and vehicle-specific access constraints. Dhaka's greater metropolitan population exceeds twenty-two million, yet its road network has not expanded at a comparable pace, making it one of the most congested large-city transportation systems in the world.

Figure I.1 shows the kind of congestion that motivates this study: buses, cars, CNG auto-rickshaws, motorcycles, and rickshaws competing for the same limited road space. Traditional navigation systems respond with static, distance-based, or at best locally reactive routing, which cannot adapt to how traffic load shifts across the network as thousands of independent trips are made in the same time window, and cannot represent the fact that different vehicle types are permitted, or practically able, to use different classes of roads.

A road network is naturally represented as a directed graph $G = (V, E)$, with V the intersections and E the directed road links between them; each link carries a length, a lane count, and a road class (arterial, collector, or local) that jointly bound its capacity. Route planning on such a graph has historically been dominated by shortest-path algorithms such as Dijkstra's algorithm [1], which are efficient and optimal for a fixed, static edge cost. Real urban traffic is not static, however: the effective cost of a link changes as vehicles enter and leave it, and a large share of trips are not independent, since they compete for the same finite capacity. Routing without accounting for this interaction causes individually shortest paths to converge onto the same attractive corridors, which saturate while parallel local and collector roads stay underused.

Three structural features make this problem acute in Dhaka. First, a small number of wide arterial corridors carry a disproportionate share of city-wide traffic alongside a dense mesh of narrow local roads. Second, the vehicle mix is extremely heterogeneous,



Figure I.1: A snapshot of severe traffic congestion on the Farmgate–Karwan Bazar corridor in Dhaka, illustrating the everyday scale of the routing problem addressed in this thesis.

from rickshaws and bicycles to CNGs, motorcycles, cars, and buses, each with different space requirements and access permissions. Third, Dhaka lacks a widely deployed, real-time, capacity-aware route-guidance system. Ant Colony Optimization (ACO) [2] addresses these features naturally: because it maintains a population-based, probabilistic search rather than a single deterministic computation, it can fold road capacity, current load, and vehicle-class access directly into its heuristic and pheromone mechanisms, making it a suitable foundation for the framework developed in this thesis.

1.2 Problem Statement

The problem addressed is that route-planning methods optimize individual paths without considering network congestion and load distribution. When millions of individually optimized trips are aggregated, a corridor that appears shortest attracts a disproportionate share of traffic, which raises its cost, yet the corridor keeps being selected – producing the gridlock of Figure I.1 and its costs in lost time, fuel, emissions, and delayed emergency response.

Formally, given a directed graph $G = (V, E)$ for Dhaka, heterogeneous vehicle classes with Passenger Car Unit (PCU) weights, a road-class access matrix, and 500 origin–destination queries $q_1, \dots, q_{500}$ (each with a source, destination, vehicle type, count, and priority), the problem is to assign every query a path such that: (i) the path is feasible under the access matrix; (ii) the total routing cost, accounting for link congestion, is

minimized; (iii) average network congestion is reduced relative to independent routing; and (iv) traffic load is spread evenly across links. Classical shortest-path and basic ACO methods satisfy (i) but generally fail (ii)–(iv) because they route each query independently against a congestion-blind cost. This thesis investigates whether a modified road-cost function combined with priority ordering and pheromone discouragement can measurably improve cost, congestion, and load balance over a congestion-unaware ACO baseline on the same Dhaka dataset and query set.

1.3 Research Gap

No reviewed study integrates road-class access constraints, PCU-based load, capacity-aware cost, and priority-based pheromone discouragement in one routing framework at Dhaka scale. Prior work supplies individual building blocks – road-condition-aware ACO heuristics [3], AHP-weighted cost factors [4], and combined guidance/signal-timing adaptation [5] – but, as Chapter II shows and this thesis verifies by re-implementing three closely related methods on its own dataset, these either treat queries independently, ignore vehicle-class access, or omit network-wide load balance.

1.4 Research Aim

The aim is to reduce network-level congestion in megacity Dhaka by developing and evaluating an adaptive, congestion-aware route-optimization framework based on improved Ant Colony Optimization, in place of purely individual, distance-based routing, so that road class, capacity, current load, and vehicle-class access jointly guide routes for many simultaneous, heterogeneous queries.

1.5 Research Objectives

The objectives of this research are as follows:

1. To model the central Dhaka road network as a directed graph with realistic node, link, road class, lane-count, and length attributes, and represent the vehicle fleet through PCU weighting and a road-class access matrix.
2. To formulate a multi-factor, congestion-aware road-cost function combining distance, lane-based capacity, traffic load, and access, and use it as the ACO heuristic.
3. To design and implement Independent ACO (baseline) and Priority-Discouragement ACO (proposed), and apply both to the same 500-query Dhaka dataset under identical parameters.

4. To compare the two strategies using total cost, average congestion, and load balance, and verify the cost model through a manual path-evaluation case study.
5. To re-implement closely related published methods on the same dataset and compare all methods through a common evaluation layer reporting identical metrics without altering any original algorithm.

1.6 Significance of the Study

This study is significant methodologically, practically, and infrastructurally. Methodologically, it contributes a multi-factor road-cost model and a novel Priority-Discouragement ACO strategy (Chapter III, novelty argument in Section 3.9), together with a fair, re-implementation-based comparison methodology – three related methods re-implemented on the same Dhaka dataset and compared through a common evaluation layer, reusable for future studies.

Practically, the framework can serve as the routing engine of a congestion-aware navigation or ride-hailing application that steers traffic away from saturated corridors; its cost model and load-balance metric can help identify structurally overloaded links for infrastructure or signal-timing intervention; and its priority mechanism applies directly to emergency-vehicle routing, where ambulances and fire services are processed first and protected from congestion caused by lower-priority traffic routed afterward.

In summary, the study contributes: (1) a cleaned Dhaka road-network dataset (1,168 nodes, 3,473 directed links) with matching vehicle, access-matrix, traffic-flow, and query data; (2) a congestion-aware road-cost model with a congestion coefficient and path-capacity calculation; (3) two ACO-based routing strategies; (4) a quantitative comparison on 500 heterogeneous queries, supported by a manual case study; and (5) a common evaluation layer placing the proposed method and five re-implemented method families on identical metrics.

1.7 Scope of the Study

The study covers dataset preparation, road-cost modeling, ACO-based routing, priority-aware assignment, and comparison on selected network-level metrics, implemented as a custom C++ framework without commercial traffic-simulation or GIS software. The network is restricted to central Dhaka (roughly $17 \text{ km} \times 28 \text{ km}$, spanning Motijheel, Dhanmondi, and Tejgaon), comprising 1,168 nodes and 3,473 directed links. Five vehicle categories are modeled (motorcycle, CNG, private car, bus, rickshaw/bicycle); rail, waterway, and pedestrian modes are out of scope. Link traffic load is generated by a Poisson-based synthetic inflow/outflow model calibrated by road class, rather than live

sensor or GPS data. Evaluation uses the 500-query dataset and covers total cost, average congestion, and load balance, plus the common-layer metrics (success rate, runtime, Jain’s fairness index); it excludes full microscopic simulation (car-following, signal phasing) and field deployment, both left for future work (Chapter VII).

1.8 Limitations of the Study

The main limitations follow from synthetic traffic data, a bounded central-Dhaka study area, and a single-snapshot offline evaluation: traffic is generated by a calibrated stochastic model rather than measured; the network excludes the full metropolitan area; both methods are evaluated against one simulated time step; parameters are fixed rather than tuned; and 149 of the 500 generated queries are infeasible on the cleaned dataset (disconnected under the access constraints), limiting the effective demand routed by both methods. Consequences for interpreting the results are examined in Section 7.3.

1.9 Thesis Organization

Chapter II reviews prior work and derives the research gap. Chapter III describes the research framework, data preparation, the cost model, the two ACO-based routing strategies, the experimental design, and the novelty argument. Chapter IV presents the experimental setup, results, and comparative analysis. Chapter V discusses engineering, ethical, and professional considerations. Chapter VI maps the work to complex engineering problems and activities. Chapter VII summarizes findings, limitations, and future work. The report closes with IEEE-format references and an appendix of source-code excerpts and reproduction instructions.

The work was carried out over two academic terms following the week-by-week plan in Figure I.2: the first term covered literature review, dataset preparation, road-network modeling, and access-matrix creation; the second covered cost modeling, both ACO methods, evaluation, re-implementation of related methods, and reporting. The thesis was carried out by a single author (Sourav Debnath, Roll: 2007109) under the supervision of Dr. Muhammad Aminul Haque Akhand, with periodic review and guidance at each stage.

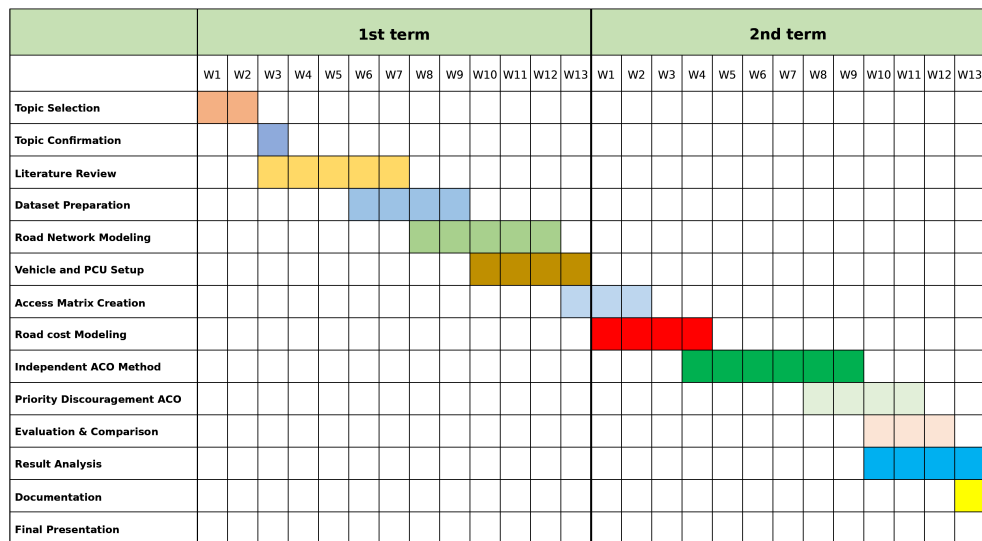


Figure I.2: Gantt chart showing the week-by-week project plan across the first and second terms of the thesis.

CHAPTER II

Literature Review

Congestion-aware route optimization draws on road-network modeling, traffic-congestion theory, and computational intelligence, and the studies reviewed here – reinforced by the author’s own re-implementation of five related methods on the same Dhaka dataset – converge on a specific, verifiable gap that motivates the proposed method.

2.1 Background of the Problem

A road network is modeled as a graph $G = (V, E)$, with nodes as intersections and edges as directed links carrying length, lane count, and road class. *Network design* problems decide which links to build or modify (e.g., the bicycle-lane design of Section 2.4.4), while *route assignment* problems, the focus of this thesis, decide how existing demand is assigned to a fixed network. Bader *et al.* (2011) [6] show that jointly optimizing a compact set of good *alternative* routes (rather than one shortest path) is NP-hard and propose penalty-based heuristics; the multi-candidate-path generation used here is conceptually aligned but couples candidates to a congestion-aware cost and a multi-query load-balancing objective.

Congestion is commonly modeled through volume-to-capacity ratios – the Bureau of Public Roads (BPR) function [7], $t = t_0(1 + \alpha(v/c)^\beta)$, used in Zhang *et al.* [5] – or through a coefficient derived from vehicle space versus available road space per lane, the approach used by Wu *et al.* [3] and adopted for the cost model of Section 3.5. Since continuous real-time sensing is unavailable for most of Dhaka, this thesis estimates traffic load with a Poisson-distributed synthetic inflow/outflow process calibrated by road class, as most reviewed studies also do in the absence of live data.

Load balancing distributes traffic across alternative links so that no corridor is overused while parallel roads sit idle; it is commonly measured through the coefficient of variation (CV) of link utilization or Jain’s fairness index [8]. This thesis reports both: load balance as $1/(1 + CV)$ (Section 3.8) and Jain’s index through the common evaluation layer (Section 3.8). None of the four directly related studies reviewed below reports an explicit network-wide load-balance metric – part of the gap identified in Section 2.6.

2.2 Significance of Computational Intelligence in Traffic Route Optimization

Congestion-aware, multi-query routing is a nonlinear, interacting-decision problem that exact shortest-path methods represent poorly, since the cost of one decision depends on concurrent decisions elsewhere. Metaheuristics such as Ant Colony Optimization [9], Particle Swarm Optimization [10], genetic algorithms, and simulated annealing perform guided stochastic search and accommodate non-additive, multi-objective costs more naturally than exact methods. This thesis adopts ACO both for its natural interpretation as an incremental, distributed route-building process and its demonstrated effectiveness in prior traffic-routing work (Section 2.4).

In the canonical formulation [2], [11], ant k at node i selects the next node j with probability

$$p_{ij}^k = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{s \in \mathcal{N}_i^k} (\tau_{is})^\alpha (\eta_{is})^\beta}, \quad (\text{II.1})$$

where τ_{ij} is pheromone, $\eta_{ij} = 1/d_{ij}$ is the classical distance-only heuristic, and α, β weight pheromone versus heuristic influence. After each tour, pheromone evaporates at rate ρ and is reinforced on good tours, $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_k \Delta\tau_{ij}^k$, with $\Delta\tau_{ij}^k \propto 1/L_k$. Because $\eta_{ij} = 1/d_{ij}$ carries no information about capacity, load, or access, basic ACO applied directly to routing behaves like a stochastic shortest-path method with the same congestion-blind limitation (Section 2.4.1).

CI-based routing offers three advantages exploited in this thesis: arbitrary desirability factors (congestion, capacity, access) fold into the heuristic without changing the search algorithm (Section 3.5); shared pheromone memory lets routing decisions influence one another across ants and, here, across successive queries (Section 3.6.2); and population-based search yields several diverse candidate paths per query, enabling capacity-aware allocation across alternatives (Section 3.6.1). A Dijkstra fallback over the same cost function guarantees feasibility whenever the stochastic search fails, combining CI flexibility with exact-method robustness. Unlike supervised learning, ACO requires no training corpus, only a network graph, a traffic-state representation, a demand set, and calibration constants – all satisfied for Dhaka as described next.

2.3 Benchmark Dataset and Procedure of Data Collection

No public benchmark exists for Dhaka-scale, vehicle-class-aware route optimization; reviewed studies instead use small synthetic networks (e.g., Wu *et al.*'s 38-node Beijing case) or non-reusable, author-specific networks, making cross-paper comparison unreliable. This thesis digitizes central Dhaka into a structured dataset (1,168 nodes, 3,473 directed links after cleaning) with matching vehicle, access-matrix, traffic-flow,

and query data (Section 3.3). The same dataset is used for the proposed method *and* for the author’s re-implementations of five related published methods (three ACO routing methods, one signal-timing study, and one network-design study), scored through a common evaluation layer (Section 3.7.3) so cross-method observations in Sections 2.6–2.7 reflect the same network and demand rather than differences between each paper’s own test case.

2.4 Review of Previous Studies

Prior work falls into shortest-path/heuristic methods, improved-ACO methods, combined/hybrid methods, and Dhaka-specific studies. Four directly related works – Wu *et al.* [3], Cheng [4], Zhang *et al.* [5], and Akhand *et al.* [12] – were additionally re-implemented by the author on the Dhaka dataset (Section 2.6), along with a signal-timing-only study, for a total of five re-implementations feeding Chapter IV.

2.4.1 Shortest-Path and Heuristic Methods

Dijkstra’s algorithm [1] computes minimum-cost paths in $O(|E| + |V| \log |V|)$ time [13] and, with variants such as A* and bidirectional search, remains the dominant exact method; both ACO methods in this thesis fall back to it when the stochastic search fails (Section 3.6.1). Because its edge cost is fixed per query and queries are solved independently, it cannot represent how routing one set of vehicles changes conditions for those routed afterward. Heuristic refinements combine multiple factors – road width, lanes, historical congestion – via rule-based weighting or the Analytic Hierarchy Process (AHP) [14]; Cheng (2023) [4] combines normalized distance, time, and a lane-based traffic term with AHP weights (0.637, 0.258, 0.105). Such fixed, pre-computed weightings do not adapt as network-wide congestion evolves during routing, a gap the priority-discouragement design addresses.

2.4.2 Improved ACO Methods

Ant Colony System [11] and MAX–MIN Ant System [15] improve convergence through modified pheromone rules; a larger body of work instead modifies the heuristic itself. Wu, Zhou, and Xiao (2020) [3] propose a road-condition factor $R_i(t) = d_i(1 + C_i(t))$, using $\eta_{ij} = 1/R_{ij}$ in place of $1/d_{ij}$ and tuning α, β, ρ with PSO [10]; on a 38-node Beijing network they report congestion-reduction ratios of 8.91–15.61% (static) and 13.28–18.20% (ACO). This road-condition idea is the starting point for the cost model of this thesis, extended with vehicle-class access and PCU-weighted load. Cheng (2023) [4] adds a destination-directed term to the AHP-weighted cost and a dynamic pheromone-rescaling rule for temporal changes in traffic state between runs – distinct from the *within-run, cross-query* discouragement proposed here.

Beyond these two, recent work refines ACO along complementary axes without addressing simultaneous, access-restricted, multi-query routing: Yu (2024) [16] adds a taboo-table rollback and A* stage for single-trip green routing (22.07% shorter paths); Wei *et al.* [17] apply Softmax-guided ACO to bus-network design rather than per-query routing; Lee *et al.* [18] embed ACO in a vehicular-network communication protocol; and Ghaffari *et al.* [19] jointly optimize congestion and air pollution, reflecting growing interest in multi-objective road costs. A further group confirms that biasing ants away from over-used edges is an active, effective direction: Zhu *et al.* [20] add a reward/punishment pheromone term (analogous to discouragement, but for single-route planning); Wang [21] and Korzeń and Kruszyna [22] redefine the objective for low-carbon logistics and public-transport planning respectively; Liu and Wu [23] hybridize ACO with a genetic algorithm for tourism routing; and Arpitha *et al.* [24] select multiple low-overlap paths for IoT source-location privacy, a multi-path formulation similar in spirit to the several-candidates- per-query design used here. From the same supervisory research line, Shahadat *et al.* [25] propose an adaptive-visibility ACO and Akhand *et al.* [26] a discrete spider-monkey variant, both for the Travelling Salesman Problem – confirming heuristic/pheromone modification as the principal lever for improving ACO, but none applies it to priority-ordered, access-restricted, multi-query urban routing.

2.4.3 Combined and Hybrid Methods

Classical traffic assignment [27], [28] (User Equilibrium, System Optimum) distributes demand without distinguishing trip importance. Zhang *et al.* [5] instead combine BPR route guidance with Webster/PSO signal-timing optimization for accident-affected networks, comparing no-optimization, guidance-only, signal-only, and combined strategies; though not ACO-based, its idea of an explicit priority order underlies the query-importance ordering of Section 3.6.2. Labib *et al.* [29] optimize signal timing alone for a Dhaka intersection ($\sim 40\%$ queue reduction) without any origin–destination routing. Emergency-vehicle studies develop priority explicitly: Musolino *et al.* [30] integrate within-day dynamic assignment with short-term travel-time forecasting to design emergency-vehicle routes; Yan *et al.* [31] solve rescue-path planning with a prioritized deep Q-network; and the review by Hao *et al.* [32] stresses that priority routing must also account for its impact on ordinary traffic – the load-balancing concern the discouragement mechanism addresses. Liu *et al.* [33] couple path planning with intersection resource allocation in a 6G-driven scheme, and Lämmer and Helbing [34] show decentralized, priority-based signal self-control produces emergent green waves. These methods confirm the value of priority-aware coordination but couple it to signal or communication control rather than encoding it inside a single ACO route-assignment loop.

2.4.4 Dhaka City Road-Network-Based Studies

Akhand *et al.* (2021) [12] design a Physarum-inspired bicycle-lane network for congested Dhaka, evaluated by comparing travel time against existing motorized routes; conducted by the same supervisor, it demonstrates nature-inspired optimization’s value for Dhaka but targets network *design*, not vehicle-class-aware route assignment. Re-implemented on this thesis’s dataset (deriving synthetic pressure/demand features from available road attributes since footpath and marketplace-density fields are unavailable), it connected 120 synthetic demands with a 391.5 km lane network, reproducing the paper’s qualitative conclusion. The same research line applied PSO to Dhaka’s transportation network-*design* problem [35], underscoring the difficulty of scaling metaheuristics from small benchmarks to a full megacity graph – the same challenge confronted here from the routing side. Other Dhaka studies ground the dataset built for this thesis: Ahad *et al.* [36] build a real-time congestion analyzer using live map-service queries rather than an optimizable network model; Sabbir [37] derives PCU values for Dhaka’s mixed vehicle fleet by manual survey, directly validating the PCU weighting used in Section 3.4; Hasinae-Jannat [38] documents pedestrian road-crossing behavior at Dhaka intersections and mid-block sections, underscoring the mixed motorized/non-motorized conflict this thesis’s road-class access matrix must accommodate; Hossain *et al.* [39] use ACO for post-disaster base-station recovery in vulnerable Dhaka areas; and Sultana *et al.* [40] combine graph convolutional networks with A* for emergency-vehicle routing in Khulna. Together these confirm severe, structurally uneven congestion across Bangladeshi megacities, informing the road classification and capacity model adopted here (Section 3.3).

2.5 Summary of Existing Methods

Table II.1 summarizes the four studies most directly related to this thesis and the shortcoming the proposed framework addresses.

Three patterns emerge. In *demand model*, reviewed methods optimize a single trip (Cheng), a modest independent set (Wu: 50 pairs), demand around one disruption (Zhang), or no trips at all (signal-timing/design work) – none jointly optimizes hundreds of interacting queries. In *traffic representation*, only Wu *et al.* model a physical congestion coefficient, and none models a heterogeneous, access-restricted vehicle fleet. In *evaluation*, each paper reports its own headline metric on its own network and none reports network-wide load balance – resolved here through same-dataset re-implementation and a common evaluation layer.

Table II.1: Summary of directly related works and their shortcomings relative to this thesis.

Study	Method	Applied to	Shortcoming addressed by this thesis
Akhand <i>et al.</i> (2021) [12], <i>Applied Sciences</i>	Physarum-inspired bio-network design	Bicycle lane network design for Dhaka	Designs infrastructure, not route assignment for existing heterogeneous vehicle traffic; no PCU load or access-matrix modeling
Cheng (2023) [4], <i>J. Advanced Transportation</i>	AHP-weighted ACO (distance, time, traffic)	General road network, small OD test cases	No priority-based handling of simultaneous queries; heuristic improves convergence, not network-wide load balance
Zhang <i>et al.</i> (2023) [5], <i>J. Advanced Transportation</i>	BPR route guidance + Webster/PSO signal timing	Accident-affected urban network	Not ACO-based; no vehicle-class access modeling; optimizes travel time/delay, not total cost and load balance jointly
Wu <i>et al.</i> (2020) [3], <i>IEEE Access</i>	Improved ACO with road-condition factor	General traffic network (Beijing case study, 38 nodes)	No priority-based traffic handling; reported benefit does not scale to Dhaka-sized network (re-verified in this thesis); no vehicle-class access matrix
Labib <i>et al.</i> (2019) [29], <i>Urban Science</i>	Data-mining + microsimulation signal timing	Signalized intersection in Dhaka	Optimizes signal timing only; performs no origin–destination routing and no network-wide vehicle assignment

2.6 Critical Analysis of Previous Work

To move beyond a literature-only critique, Wu *et al.* [3], Cheng [4], and Zhang *et al.* [5] were independently re-implemented in C++ and executed on the same central-Dhaka network used throughout this thesis (a signal-timing study and the Physarum design were also re-implemented; all five feed Chapter IV).

Wu *et al.*: benefit shrinks at Dhaka scale. Over 34 feasible of 50 origin–destination pairs, the re-implementation reproduced the paper’s static and ACO congestion-reduction ratios reasonably closely (12.48% and 18.33% versus reported 8.91–15.61% and 13.28–18.20%), but its PSO-tuned stage diverged sharply ($CR_{\text{opt}} = 10.60\%$, promotion rate 6.54%, versus reported 2.08–2.23% and 9.73–13.63%), attributable to Dhaka’s roughly $30\times$ larger scale and different congestion calibration. The road-condition-factor idea is sound but incomplete: it does not by itself scale its benefit to Dhaka and lacks

vehicle-class access or priority.

Cheng: heuristic refinement improves convergence, not load distribution. On one representative pair (node 1 to node 100, 1,061-node component), the improved ACO found a 6-hop, 20.30 km route (cost 2.0537) versus a slightly shorter 5-hop, 19.65 km route (cost 1.9789) from basic ACO and Dijkstra baselines – showing that an AHP-weighted heuristic mainly affects which near-optimal path is found, not network-wide load distribution, since no mechanism lets one query’s decision influence another’s.

Zhang *et al.*: combined guidance trades cost against time, ignores heterogeneity. With a simulated closure on a 1,168-node, 7,124-link network (Webster timing for 707 intersections), the combined strategy (D) cut travel time from 898.3 to 836.9 s and delay from 314.0 to 264.2 s versus the unoptimized baseline (A), yet signal-timing alone (C) achieved lower total cost (177,656 vs. 178,251), since detour rerouting raised distance cost. Network-wide utilization barely moved (0.563 across all four methods), confirming the method redistributes traffic only locally.

Across all reviewed work, three weaknesses recur: queries are routed independently or around a single disruption, so no method coordinates hundreds of simultaneous trips; vehicle heterogeneity (footprint, legal road-class access) is unmodeled; and evaluation uses each paper’s own metric with no common, network-wide load-balance basis. These weaknesses, verified empirically rather than assumed, define the research gap.

2.7 Identified Research Gap

No re-implemented method – ACO-based [3], [4], [5] or the bicycle-lane design [12] – combines: (i) a road-class access matrix restricting which of five vehicle types may use each road class; (ii) a PCU-weighted, lane-based capacity model; (iii) a capacity-aware cost embedded directly in the ACO heuristic; and (iv) priority discouragement that orders queries by importance and steers demand away from links already used by higher-priority queries. This four-way integration, evaluated on a 1,168-node, 3,473-link Dhaka network with 500 heterogeneous queries and compared against the re-implemented methods through a common evaluation layer, is the specific, verifiable gap this thesis addresses.

2.8 Conceptual Framework and Proposed Research Direction

The framework treats route optimization as a closed loop: traffic state shapes route cost, cost guides ant-based search, and completed assignments update the state for subsequent queries. Four components interact – a *data layer* (road graph, PCU-weighted vehicle classes, access matrix, synthetic flows, prioritized queries), a *cost layer* converting traffic state into per-link, per-vehicle-class cost, a *search layer* where improved ACO builds

candidate paths using that cost as heuristic, and a *coordination layer* – absent from all reviewed work – that processes queries in priority order, discourages pheromone on heavily used links, and propagates each assignment into the state seen by the next query. Two instantiations are compared: coordination disabled (Independent ACO) and enabled (Priority-Discouragement ACO). The full architecture is developed in Section 3.1 and evaluated fairly via the common layer of Section 3.7.3.

2.9 Chapter Summary

Shortest-path methods are efficient but congestion-blind; heuristic and improved-ACO methods add desirability factors but treat queries independently and ignore vehicle-class access; combined guidance/signal-timing methods introduce useful priority-ordering ideas but are not ACO-based and redistribute traffic only locally; and load-balancing, though studied generally, is not evaluated by any directly related study. These conclusions were verified by re-implementing five related methods on this thesis’s own Dhaka dataset. Chapter III builds on this review to develop a congestion-aware cost model and two ACO-based strategies that jointly integrate road access, PCU-based load, capacity-aware cost, and priority discouragement, evaluated against the re-implemented methods through a common evaluation layer.

CHAPTER III

Research Methodology

Transforming Dhaka road-network data into optimized, congestion-aware route assignments requires a research framework, a prepared dataset, a congestion-aware cost model, two ACO-based routing strategies, an experimental design, and a set of evaluation metrics – each stated first as a formal **Rule** (its input, output, and governing equations) and then described in plain language, so that the exact quantity computed at every stage is unambiguous before it is explained.

3.1 Research Framework

The framework is a four-stage pipeline: (1) dataset collection and road-network modeling; (2) road-factor and cost modeling; (3) improved ACO-based routing, in which Independent ACO and Priority-Discountage ACO construct and select paths using the same cost model; and (4) evaluation and output generation. Figure III.1 shows the corresponding system-level, five-layer architecture realized in software, built on the complete 1,168-node, 3,473-link dataset of Table III.4.

At the software level, the framework is a modular C++ pipeline: a dataset builder, a traffic flow generator, a shared cost-model module, two ACO-based routing modules, a manual path-evaluation utility, and a metrics module, all operating over one in-memory `Dataset` loaded once from CSV.

3.1.1 Problem Formulation

The Dhaka road network is a directed graph $G = (V, E)$, $|V| = 1,168$, $|E| = 3,473$. Each link $e \in E$ has length d_e , lane count λ_e , and road type $r_e \in \{\text{arterial, collector, local}\}$. Five vehicle classes $Q = \{q_0, \dots, q_4\}$ each carry a PCU weight pcu_q and average length ℓ_q ; a road-class access matrix $A[r][q] \in \{0, 0.5, 1\}$ specifies whether class q may use road type r ; a traffic-state matrix $\text{traffic}[e][q]$ records vehicles of class q on link e . Given 500 queries $\mathcal{Q} = \{q^{(1)}, \dots, q^{(500)}\}$ (each with a source, destination, vehicle class, count, and priority), the task is to assign every query a path such that every link is access-feasible ($A[r_e][q] > 0$) and, across all queries jointly, total cost, average congestion, and load balance (Section 3.8) are favorably optimized.

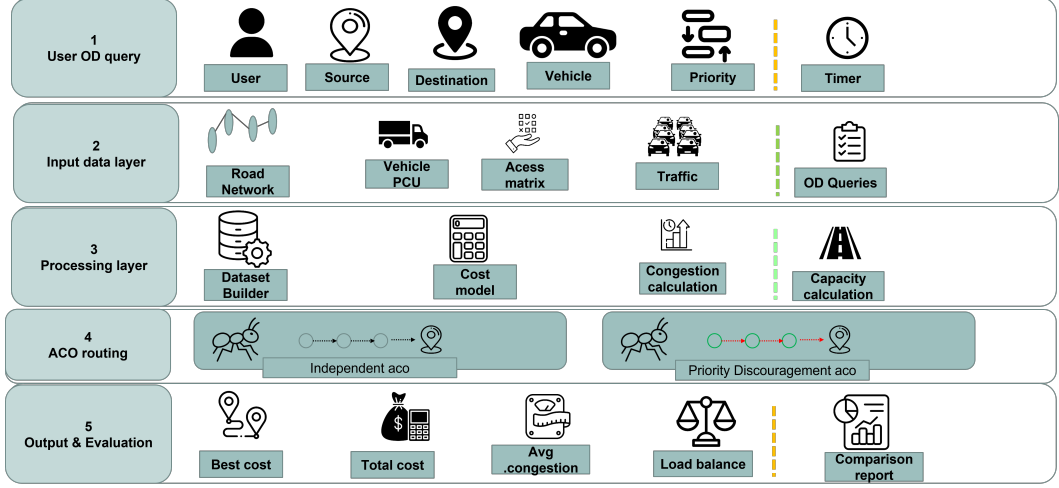


Figure III.1: Overall architecture of the proposed congestion-aware ACO route optimization system, organized as five layers: (1) user OD query, (2) input data layer, (3) processing layer, (4) ACO routing (Independent ACO and Priority-Discouragement ACO), and (5) output and evaluation.

3.2 Preliminary: Ant Colony Optimization and Its Adaptation

The proposed framework builds on the classical Ant Colony Optimization (ACO) mechanism, changing specific parts of it for congestion-aware, multi-query routing in Dhaka. Table III.3 first fixes the notation used throughout the rest of this chapter, so that every symbol introduced later refers back to a single definition.

3.2.1 Classical Ant Colony Optimization

ACO is a population-based metaheuristic inspired by the foraging behavior of real ants [2]: individual ants move over a graph leaving a chemical pheromone trail, and later ants probabilistically prefer edges with a stronger trail, so that shorter or more frequently used paths are reinforced over time while unused paths evaporate. On a graph $G = (V, E)$, an artificial ant k standing at node i chooses its next node j with the transition probability of Table III.1.

$$p_{ij}^k = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{s \in N_i^k} (\tau_{is})^\alpha (\eta_{is})^\beta}. \quad (\text{III.1})$$

After every ant completes its construction, pheromone is updated by evaporation followed by deposit, so that trails belonging to good tours are reinforced and old trails fade:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_k \Delta \tau_{ij}^k, \quad \Delta \tau_{ij}^k \propto \frac{1}{L_k}. \quad (\text{III.2})$$

This classical formulation is distance-driven: $\eta_{ij} = 1/d_{ij}$ carries no information about

Table III.1: Terms of the classical ACO transition probability, Equation III.1.

Term	Description
p_{ij}^k	Probability that ant k moves from node i to node j
τ_{ij}	Pheromone intensity currently deposited on edge (i, j)
η_{ij}	Heuristic desirability of edge (i, j) ; classically $\eta_{ij} = 1/d_{ij}$ (inverse distance)
α, β	Exponents weighting pheromone influence versus heuristic influence
$\mathcal{N}_i^k$	Set of feasible next nodes available to ant k from node i

Table III.2: Terms of the classical pheromone-update rule, Equation III.2.

Term	Description
$\rho \in (0, 1)$	Evaporation rate: fraction of pheromone lost each iteration
$\Delta\tau_{ij}^k$	Pheromone deposited on edge (i, j) by ant k , proportional to $1/L_k$
L_k	Total length (or cost) of the tour completed by ant k

road capacity, current traffic load, or vehicle-class access, and every ant is independent of every other query. Applied directly to Dhaka traffic routing, it would behave as a stochastic shortest-path method with the same congestion-blind limitation as Dijkstra's algorithm [1].

3.2.2 Adaptation for Congestion-Aware, Multi-Query Routing

Three specific changes convert the classical mechanism above into the framework used in this thesis, each implemented and evaluated in the sections that follow.

- (i) **Heuristic replacement.** The distance-only heuristic $\eta_{ij} = 1/d_{ij}$ is replaced by $\eta_e(q) = 1/(\text{cost}_e(q) + \varepsilon)$, where $\text{cost}_e(q)$ is the congestion-aware, access-permission-weighted road cost of Section 3.5 (Rule: Congestion-Aware Road Cost) rather than raw distance, so ants are guided by current network conditions and legal vehicle-class access instead of geometry alone.
- (ii) **Two comparable routing strategies.** The same adapted transition rule and pheromone mechanism are instantiated twice: Independent ACO (Section 3.6.1), which resets pheromone and uses a static traffic state per query, exactly mirroring how classical ACO treats queries as unrelated; and Priority-Discouragement ACO (Section 3.6.2), the proposed method, which keeps one persistent pheromone table across all 500 queries and orders them by an importance score $I(q)$ before routing.

Table III.3: Notation used throughout the methodology (classical ACO symbols and this thesis’s extensions).

Symbol	Description
$G = (V, E)$	Directed road-network graph; nodes V , directed links E
d_e, λ_e, r_e	Length, lane count, and road type of link e
$q \in Q$	Vehicle class (motorcycle, CNG, car, bus, rickshaw/bicycle)
pcu_q, ℓ_q	Passenger Car Unit weight and average physical length of class q
$A[r][q]$	Access permission of class q on road type r (1, 0.5, or 0)
L_e, C_e, cap_e	Road load, congestion coefficient, and PCU capacity of link e
$\text{cost}_e(q), J(P, q)$	Road cost of link e and total cost of path P , for class q
$\tau_e, \eta_e(q)$	Pheromone and (adapted) heuristic desirability of link e
$\alpha, \beta, \rho, \tau_0$	ACO pheromone/heuristic exponents, evaporation rate, initial pheromone
$I(q), w_p, w_d, d^*(q)$	Query importance score and its priority/distance weights and topological distance
ϕ_e, θ	Usage fraction of link e and discouragement strength
TC, AC, LB, $\mathcal{J}$	Total cost, average congestion, load balance, and Jain’s fairness index

- (iii) **Pheromone discouragement.** Beyond classical evaporation, links already used by higher-priority queries have their pheromone explicitly reduced in proportion to how heavily they were used (Section 3.6.2, Rule: Priority Score and Discouragement), a network-coordination mechanism absent from the classical algorithm and from every reviewed prior ACO routing method (Section 3.9).

Everywhere below, τ , α , β , and ρ retain their classical meaning (Tables III.1–III.2); only η is redefined via the cost model, and the query-ordering/discouragement layer is new. Table III.3 below collects this classical notation together with every additional symbol introduced by the adaptation, for reference throughout the remainder of the chapter.

3.3 Data Collection and Preparation

The study area is central Dhaka, roughly 17×28 km, centered on Motijheel, Dhanmondi, and Tejgaon (Figure III.2). The network was collected manually rather than from a ready-made road-network file: each major intersection was selected as a node and located with its Plus Code from Google Maps, and each road segment connecting two such intersections was recorded as a directed edge with a length estimated from the mapped route. Lane count for a given edge was estimated by dropping Google Street View’s pegman (the “yellow man”) onto that road segment and visually counting how many

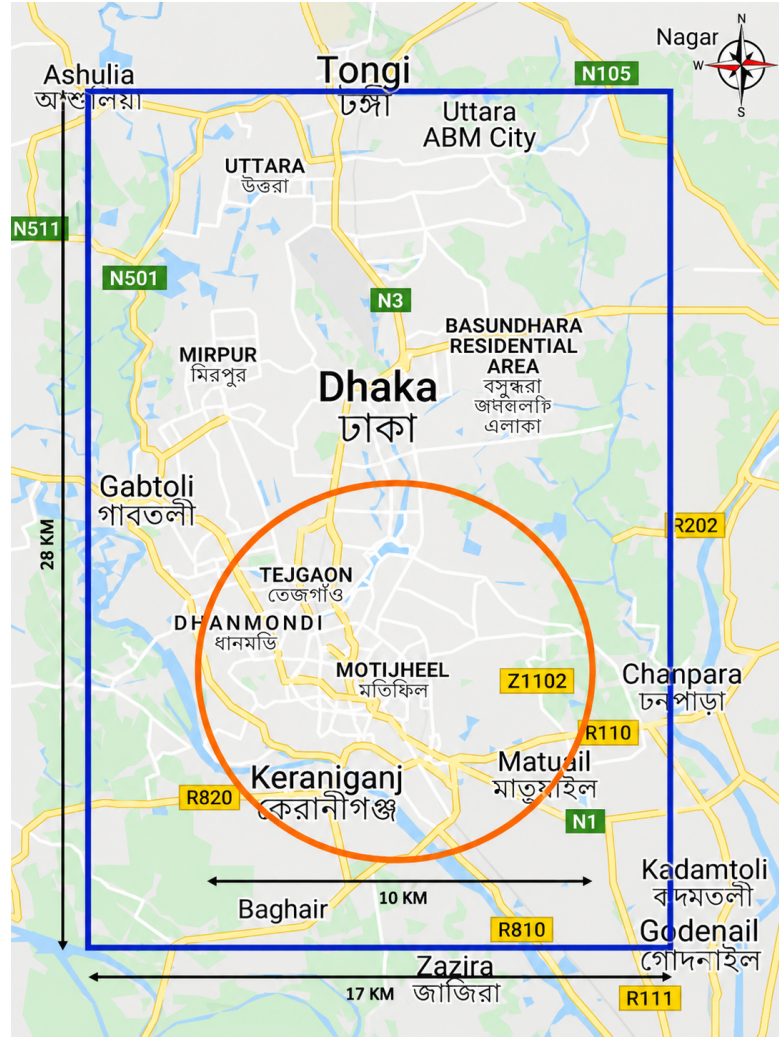


Figure III.2: Central Dhaka study area used for road-network dataset preparation, with a 10 km-radius core region (orange circle) inside a 17×28 km bounding region (blue rectangle).

vehicles the carriageway could accommodate side by side (Figure III.3), cross-checked against visible lane-divider markings. Because every node, edge, and lane count was collected by hand rather than parsed from a structured data source, individual length and lane-count values are not claimed to be 100% accurate; the resulting dataset is, however, internally consistent and dense enough to exercise the routing framework realistically, and it was densified over successive collection passes to the complete central-Dhaka coverage used here.

Links are classified into three categories consistent with Dhaka's road hierarchy – *arterial* (wide, high-capacity corridors), *collector* (medium-capacity connectors), and *local* (narrow neighborhood streets, Figure III.4) – which set each link's base flow-rate capacity and, with vehicle class, its access permission. Lane count λ_e and length d_e were estimated from street-level map inspection (counting visible lane-divider markings, as on Kazi Nazrul Islam Avenue) and are used directly in the capacity and congestion calculations



Figure III.3: Manual lane counting with Google Street View. The pegman (the “yellow man”) is dropped onto a road segment – here Kazi Nazrul Islam Avenue – and the lane count is read off by observing how many vehicles can move side by side along the carriageway, cross-checked against the visible lane-divider markings.

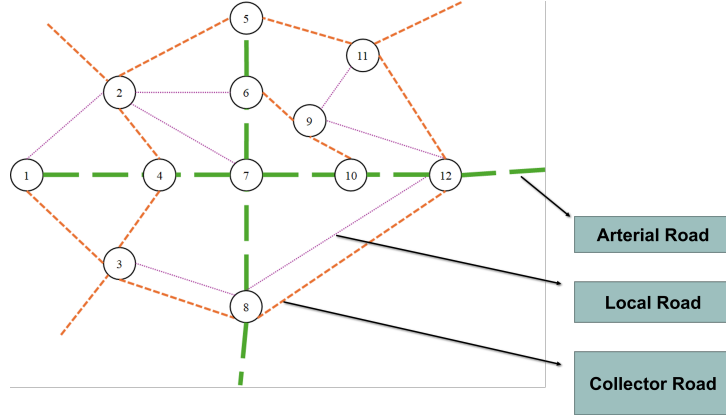


Figure III.4: Illustration of arterial, collector, and local road categories used in the access matrix.

below, not as decorative attributes.

Rule — Dataset Schema

Node: $D_V = \{(v_i, x_i, y_i)\}$ **Edge:** $D_E = \{(v_i, v_j, d_{ij}, \lambda_{ij}, r_{ij})\}$ **Vehicle:** $D_{veh} = \{(q, pcu_q, \ell_q)\}$

Accessibility: $D_A = \{(r, q, A[r][q])\}$ **Flow:** $D_F = \{(t, e, q, f_{e,q}^{in}(t), f_{e,q}^{out}(t))\}$

Dynamic traffic: $D_T = \{(t, e, q, traffic[e][q](t))\}$ **Query:** $D_Q = \{(v_s, v_d, q, n_q, \pi)\}$

This schema is realized as seven CSV files (Table III.4): nodes and links define D_V, D_E ; `vehicles.csv` and `access_matrix.csv` define D_{veh}, D_A ; `flows.csv` defines D_F, D_T over 100 time steps; `queries.csv` defines D_Q ; and `config.csv` holds the

Table III.4: Summary of input datasets used for the Dhaka road-network route optimization experiments.

Dataset file	Description	Records
dhk_nodes_clean.csv	Road-network nodes representing intersections and important points	1,168
dhk_links_clean.csv	Directed road links with length, lane count, and road type	3,473
vehicles.csv	Vehicle classes with PCU value and average length	5
access_matrix.csv	Road-type and vehicle-type access permission values	15
flows.csv	Simulated inflow, outflow, and current traffic count for each link, vehicle type, and time step	1,736,500
queries.csv	Origin–destination route-optimization queries	500
config.csv	Algorithm and simulation configuration parameters	30

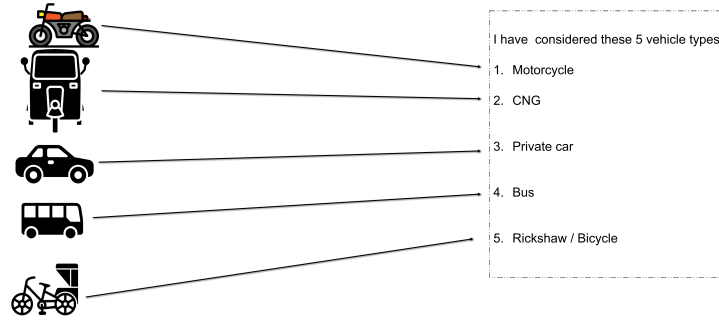


Figure III.5: Five vehicle categories considered in the PCU-based traffic model.

algorithm parameters of Table III.8.

Five vehicle categories represent Dhaka’s mixed traffic (Figure III.5), with the private car as the PCU reference class [41] (Table III.5).

Table III.6 gives the access matrix using three symbols – ✓ (permitted, $A = 1$), ? (limited, $A = 0.5$, a discouraged divisor penalty in Section 3.5), and × (prohibited, $A = 0$) – so that access rules read directly rather than as raw numbers. Rickshaws are barred from arterial roads, buses from local roads, motorcycles have unrestricted access everywhere, and several combinations (CNG on arterial, bus/rickshaw on collector) are limited.

Symbol meaning. ✓ (*permitted*) means the vehicle class may use that road type without restriction, contributing an access value $A = 1$ and no cost penalty in Equation III.7. ? (*limited*) means access is discouraged but not forbidden – for example, CNGs are legally allowed but discouraged on arterial roads – contributing $A = 0.5$, which doubles the effective road cost. × (*prohibited*) means the vehicle class may not legally use that road

Table III.5: Vehicle classes, Passenger Car Unit (PCU) weights, and average vehicle length used in this thesis.

ID	Vehicle Class	PCU	Length (m)
0	Motorcycle	0.30	2.0
1	CNG (auto-rickshaw)	0.70	3.0
2	Private car	1.00	4.5
3	Bus	2.50	10.0
4	Rickshaw / Bicycle	0.50	2.5

Table III.6: Vehicle-class access permission by road type. $\checkmark$ = permitted ($A = 1$), $?$ = limited ($A = 0.5$), $\times$ = prohibited ($A = 0$); values are used directly in the cost function of Section 3.5.

Road Type	Motorcycle	CNG	Car	Bus	Rickshaw
Arterial (highway)	$\checkmark$	$?$	$\checkmark$	$\checkmark$	$\times$
Collector (urban)	$\checkmark$	$\checkmark$	$\checkmark$	$?$	$?$
Local	$\checkmark$	$?$	$?$	$\times$	$\checkmark$

type at all (for example, rickshaws on arterial roads); the corresponding link cost is set to ∞ and the link is excluded from that class’s search entirely.

Traffic pressure on each link is represented through flow input, flow output, and current occupancy; the generation procedure is given in Section 3.4.

The 500-query demand set (Table III.7) draws source/destination uniformly from the network, vehicle class from Dhaka’s approximate modal split (motorcycle 30%, car 25%, CNG 20%, rickshaw 15%, bus 10%), vehicle count uniformly in $[25, 70]$, and priority uniformly in $[1, 5]$.

Table III.8 lists the configuration parameters used throughout the reported experiments;

Table III.7: Distribution of the 500 origin–destination queries by vehicle class.

Vehicle Class	Number of Queries	Percentage
Motorcycle	150	30.0%
Private car	131	26.2%
CNG	112	22.4%
Rickshaw/Bicycle	63	12.6%
Bus	44	8.8%
Total	500	100%

Table III.8: Key configuration parameters used for the reported experiments.

Category	Parameter	Value
Dataset mode	use_dhaka_network	1 (true)
	random_seed	42
Demand	num_queries	500
	min_vehicle_count / max_vehicle_count	25 / 70
ACO search	num_ants	20
	num_iterations	30
	num_paths (candidates retained per query)	8
	α (pheromone weight)	1.0
	β (heuristic weight)	2.0
	ρ (evaporation rate)	0.20
	τ_0 (initial pheromone)	1.0
Priority-Discouragement ACO	θ (discouragement_theta)	0.08
	w_p (gamma_priority)	0.65
	w_d (delta_distance)	0.35
Capacity model	time_interval (s)	60
	flow rate per lane – highway / urban / local (veh/hr)	1400 / 900 / 500
Flow generation	flow λ – highway / urban / local	18 / 12 / 7

their role in the cost model and ACO framework is defined in the sections that follow.

3.4 Data Preprocessing and Feature Generation

A cleaning utility (`prepare_dhaka_data.py`) removes records with non-positive length or lane count, or an unresolved endpoint, and remaps node identifiers to a dense internal index. The raw collection ($\approx 1,716$ nodes, $\approx 4,363$ links) reduces to the 1,168-node, 3,473-link “complete dataset” used throughout (Table III.9); an earlier, sparser extract of the same city is superseded by this complete version.

Arterial corridors dominate the cleaned link set (49.3%), followed by local (27.8%) and collector (22.8%) roads, and most links carry one or two lanes (Figure III.6). Out-degree ranges 0–14 (mean 2.98); 51 nodes have out-degree zero and are the dominant cause

Table III.9: Reduction from the raw collected records to the cleaned network loaded by the solver.

Element	Raw records	Loaded (cleaned)	Retained
Nodes	$\approx 1,716$	1,168	68.1%
Directed links	$\approx 4,363$	3,473	79.6%
Nodes used as a link endpoint	—	1,129	—

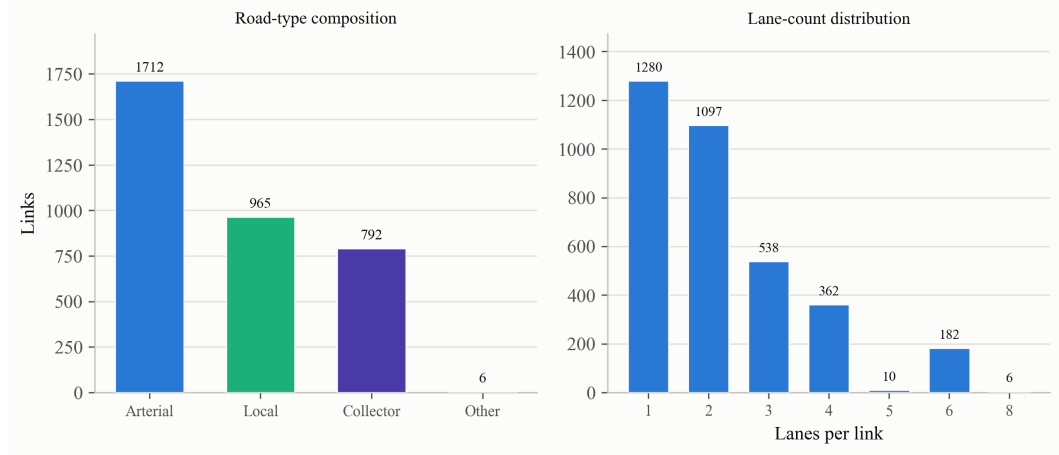


Figure III.6: Composition of the cleaned Dhaka link set: road-type distribution (left) and lane-count distribution (right). Arterial corridors dominate, and most links are one- or two-lane segments.

of the unserved queries analyzed in Section 4.6.1. Two derived quantities recur later: the network *coverage ratio* $\text{Cov} = |E_{\text{clean}}|/|E_{\text{raw}}| \approx 0.796$, and the per-link *utilization* $u_e = L_e/\text{cap}_e$ used by the load-balance measures of Section 3.8.

Rule — Traffic Flow Generation

Input: link e , vehicle class q , time t

Output: $\text{current}_{e,q}(t)$ [occupancy], $\text{pcu_load}_{e,q}(t)$

System: $\lambda_{e,q} = \lambda_{\text{base}}(r_e) w_q$; $f_{e,q}^{\text{in}}(t) \sim \text{Poisson}(\lambda_{e,q})$, $f_{e,q}^{\text{out}}(t) \sim \text{Poisson}(0.85 \lambda_{e,q})$

$\text{current}_{e,q}(t) = \max(0, \text{current}_{e,q}(t-1) + f_{e,q}^{\text{in}}(t) - \min(\cdot, f_{e,q}^{\text{out}}(t)))$

$\text{pcu_load}_{e,q}(t) = \text{current}_{e,q}(t) \times \text{pcu}_q$

Since continuous real-time sensing is unavailable for most of Dhaka, inflow and outflow are sampled independently per time step from a Poisson process calibrated by road class ($\lambda_{\text{base}} \in \{18, 12, 7\}$ for highway/urban/local) and by a per-vehicle-class weight w_q reflecting Dhaka’s relative traffic mix; outflow is capped so occupancy cannot go

negative, and any link forbidden to a vehicle class ($A[r_e][q] = 0$) has all its flow fields forced to zero. The resulting `pcu_load` is stored in `flows.csv` and feeds the cost model directly.

3.5 Congestion-Aware Cost Model

Rule — Congestion-Aware Road Cost

Input: link e , vehicle class q

Output: $\text{cost}_e(q)$ [effective travel cost of link e for class q]

System: $L_e = \sum_{q \in Q} \text{traffic}[e][q] \text{pcu}_q$ (road load)

$$C_e = \frac{\sum_{q \in Q} \text{traffic}[e][q] \ell_q}{\lambda_e d_e} \quad (\text{congestion coefficient})$$

$$\text{cap}_e = \lambda_e \phi(r_e) \frac{\Delta t}{3600} \quad (\text{PCU capacity})$$

$$\text{cost}_e(q) = \begin{cases} \infty & A[r_e][q] \leq 0 \\ \frac{d_e(1 + C_e)}{A[r_e][q]} & \text{otherwise} \end{cases}$$

This combines link length d_e with a congestion penalty $(1 + C_e)$ derived from the ratio of occupied vehicle length to available road space $(\lambda_e d_e)$, and an access-permission divisor that doubles the effective cost for partially restricted combinations ($A = 0.5$) and makes forbidden links infeasible. Figure III.7 illustrates the road-level quantities feeding this Rule – distance d_{ij} , lane count L , capacity, and the flow input/output/on-road terms – that together determine the congestion-aware travel cost of each directed link $i \rightarrow j$.

Aggregating over a path P gives the path cost $J(P, q) = \sum_{e \in P} \text{cost}_e(q)$ (infinite if any link is forbidden), and the bottleneck path capacity for class q is

$$\text{cap}(P, q) = \min_{e \in P} \left\lfloor \frac{\max(0, \text{cap}_e - L_e)}{\text{pcu}_q} \right\rfloor. \quad (\text{III.8})$$

This road cost, not raw distance, becomes the ACO heuristic in Section 3.6.

Worked example. For an arterial link ($d_e=450$ m, $\lambda_e=3$) carrying 40 motorcycles, 10 CNGs, 25 cars, 4 buses, and 6 rickshaws (Figure III.8): $L_e = 40(0.30) + 10(0.70) + 25(1.00) + 4(2.50) + 6(0.50) = 57.0$ PCU; occupied length = $40(2.0) + 10(3.0) + 25(4.5) + 4(10.0) + 6(2.5) = 277.5$ m against road space $3 \times 450 = 1,350$ m, so $C_e \approx 0.2056$; capacity $\text{cap}_e = 3 \times 1400 \times (60/3600) = 70.0$ PCU. A private car ($A=1.0$) pays $\text{cost}_e = 450(1.2056) \approx 542.5$ (a 20.6% congestion penalty over the uncongested 450), while a rickshaw ($A=0$ on arterial) faces $\text{cost}_e = \infty$ and is never

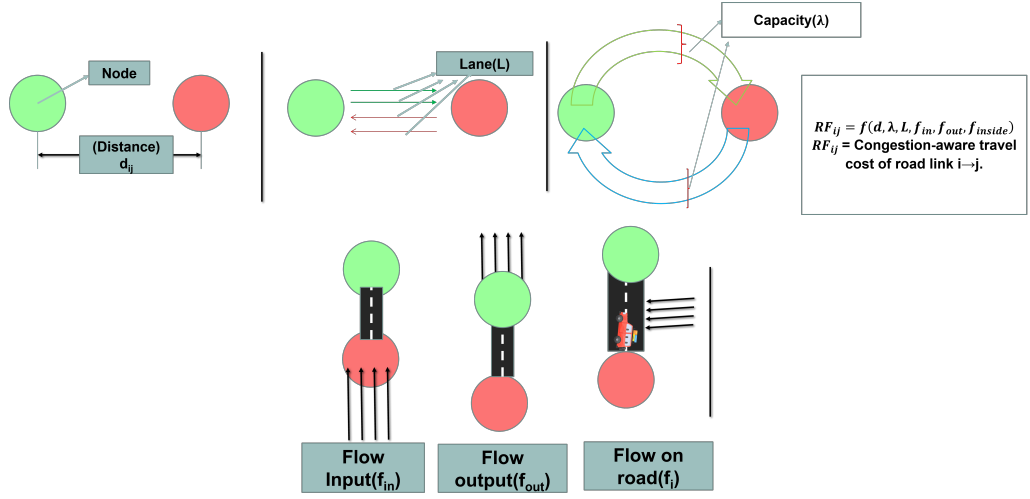


Figure III.7: Components of the road-factor / congestion-aware cost model: node-to-node distance d_{ij} , lane count, road capacity, and the flow-input/flow-output/flow-on-road quantities used in Equations III.5–III.7.

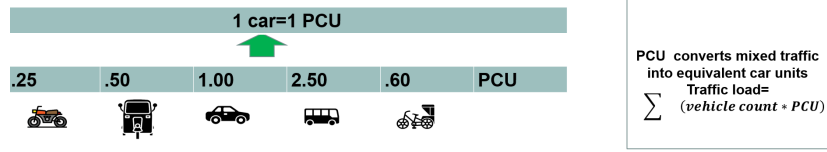


Figure III.8: Illustration of Passenger Car Unit (PCU) conversion for heterogeneous vehicle types.

routed onto this link. The remaining headroom $70.0 - 57.0 = 13.0$ PCU admits 13 more cars but only 5 more buses, showing why path capacity is vehicle-class dependent even on an identical physical route.

3.6 Improved Ant Colony Optimization Framework

Both routing methods share the ACO path-construction and pheromone-update mechanism below; they differ only in pheromone scope, query order, and whether discouragement is applied (Sections 3.6.1–3.6.2).

Rule — ACO Path Construction

Input: query (s, d, q) , pheromone τ , traffic state T

Output: candidate path P from s to d

System: $\eta_e(q) = \frac{1}{\text{cost}_e(q) + \varepsilon}$ (heuristic, replaces $1/d_{ij}$)

$$P_e^k = \frac{(\tau_e)^\alpha (\eta_e(q))^\beta}{\sum_{e' \in \mathcal{N}^k} (\tau_{e'})^\alpha (\eta_{e'}(q))^\beta} \quad (\text{transition probability})$$

Every link starts at $\tau_e = \tau_0 = 1.0$. Each of 20 ants builds a path step by step from the source ($\max \maxSteps = |V| \times 2 + 5 = 2,341$), drawing the next link by roulette-wheel sampling over P_e^k among feasible outgoing links $\mathcal{N}^k$, with $\alpha=1.0, \beta=2.0$ weighting pheromone versus the congestion-aware heuristic. If no ant reaches the destination after all iterations, an exact Dijkstra fallback [1] over $\text{cost}_e(q)$ guarantees a path whenever one exists in the access-constrained subgraph. This directly replaces the classical distance-only heuristic [2] reviewed in Section 2.2, extending the road-condition-factor idea of Wu *et al.* [3] with vehicle-class access.

Rule — Pheromone Evaporation and Update

Input: iteration's ant paths, evaporation rate ρ

Output: updated pheromone τ

System: $\tau_e \leftarrow \max(10^{-9}, (1 - \rho)\tau_e)$; $\tau_e \leftarrow \tau_e + \Delta\tau_e$, $\Delta\tau_e = \sum_{P \in \text{elite}} \frac{1}{J(P, q) + \varepsilon}$
for $e \in P$

Pheromone evaporates at $\rho = 0.20$ each iteration, then the best (lowest-cost) up to five paths deposit pheromone in inverse proportion to their cost, reinforcing links used by good paths for later iterations and, in Method 2, for later queries. After 30 iterations, all valid candidates are pooled, deduplicated, sorted by ascending cost (capacity as tie-breaker), and the best 8 are kept for vehicle allocation. Algorithm 1 summarizes this shared procedure.

Algorithm 1 GENERATEPATHS – shared ACO path-construction procedure

Require: query q (source s , destination d , vehicle class v), pheromone table τ , traffic state T , dataset $G = (V, E)$

Ensure: up to 8 candidate paths, sorted by ascending cost

```
1: for iter  $\leftarrow$  1 to 30 do
2:   for ant  $\leftarrow$  1 to 20 do
3:     path  $\leftarrow$  CONSTRUCTANTPATH( $s, d, v, \tau, T, G$ )  $\triangleright$  Eq. transition rule
4:     if path reaches  $d$  then
5:       add to candidates
6:     end if
7:   end for
8:   evaporate  $\tau$ ; deposit pheromone on the best  $\min(5, \cdot)$  paths of this iteration
9: end for
10: if no candidate found then
11:   fall back to Dijkstra over  $\text{cost}_e(q)$ 
12: end if
13: deduplicate, sort by ascending cost, return best 8
```

The per-query cost is $O(30 \times 20 \times 2,341 \times \Delta_{\text{avg}}^+)$ ($\Delta_{\text{avg}}^+ \approx 2.97$ average out-degree), plus $O(|E| + |V| \log |V|)$ when the Dijkstra fallback triggers; Method 2 adds only an $O(|E|)$ discouragement pass and an $O(500 \log 500)$ priority sort per run, so both methods scale similarly at this network size. Raising ants/iterations to the code’s higher defaults (80/80) would increase runtime roughly $16\times$, motivating the smaller 20/30 configuration used for the reported experiments.

3.6.1 Method 1: Independent ACO-Based Route Optimization

Independent ACO processes each of the 500 queries separately as a baseline (Figure III.9): pheromone is reinitialized to τ_0 for every query, the static initial traffic state T_0 is used throughout (routing one query does not affect the cost seen by the next), and up to 8 candidate paths are generated per query. Candidates are sorted by ascending cost and the query’s vehicle count is filled greedily across them up to each path’s capacity (Equation III.8), lowest-cost path first; any demand exceeding total candidate capacity is still assigned to the cheapest path (an “always serve” policy). After all queries are processed, every assignment is replayed onto T_0 to obtain the final traffic state used for metrics. Algorithm 2 summarizes the method.

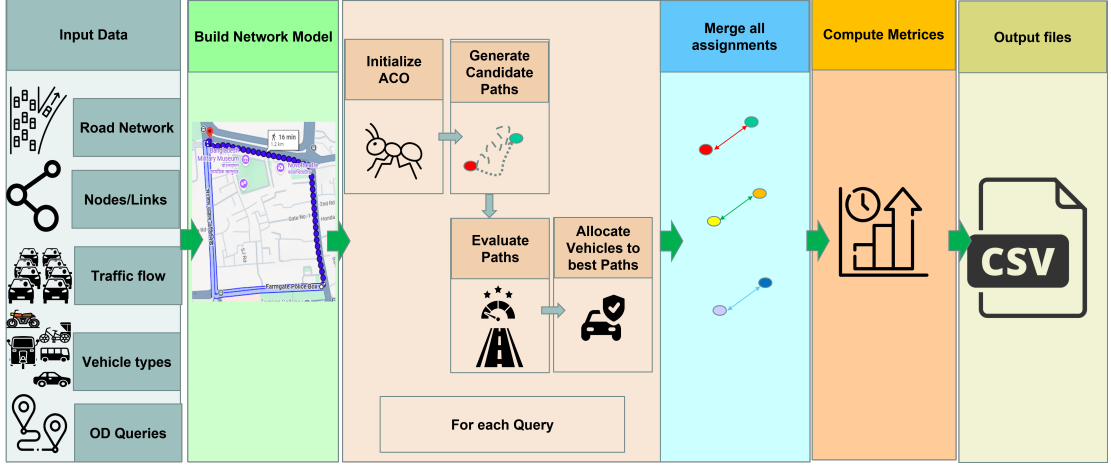


Figure III.9: Independent ACO workflow (Method 1): for each query, the network model and candidate paths are built and evaluated independently, vehicles are allocated to the best paths, and assignments are merged before metrics are computed. Route selection relies solely on the pheromone and heuristic values, with no cross-query interaction.

Algorithm 2 Method 1 – Independent ACO

Require: dataset G , static traffic state T_0 , queries $Q = \{q_1, \dots, q_{500}\}$ in original order

Ensure: route assignments and final traffic state T_{final}

- 1: **for each** $q_i \in Q$ in original order **do**
 - 2: reinitialize $\tau \leftarrow \tau_0$
 - 3: paths $\leftarrow \text{GENERATEPATHS}(q_i, \tau, T_0, G)$
 - 4: allocate q_i 's vehicles greedily across paths by ascending cost
 - 5: **end for**
 - 6: replay all assignments onto T_0 to obtain T_{final} ; **return** assignments, T_{final}
-

3.6.2 Method 2: Priority-Discouragement ACO-Based Route Optimization

Priority-Discouragement ACO (Figure III.10) reorders queries by importance and discourages pheromone on links already loaded by higher-priority queries.

Rule — Priority Score and Discouragement

Input: query q , its assigned paths

Output: importance $I(q)$; discouraged pheromone τ_e on links used by q

System: $I(q) = w_p \cdot \text{priority}(q) + w_d \cdot \frac{1}{d^*(q) + \varepsilon}$

$$\phi_e = \min\left(1, \sum_{P \ni e} \frac{\text{assigned}(P)}{\max(1, \text{vehicleCount}(q))}\right); \quad \tau_e \leftarrow \max(10^{-9}, \tau_e(1 - \theta \phi_e))$$

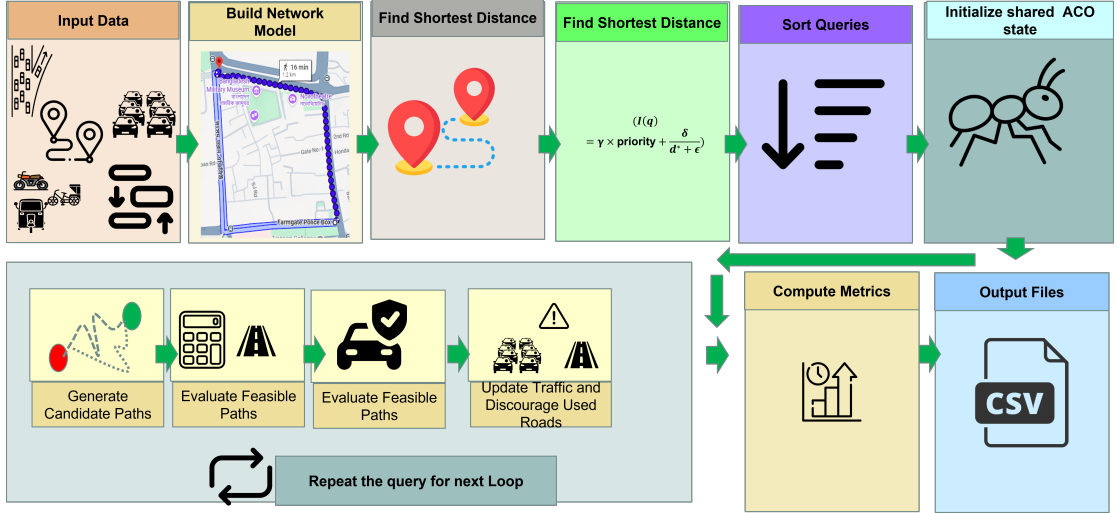


Figure III.10: Priority-Discouragement ACO workflow (Method 2, proposed): queries are sorted by importance $I(q)$ and a shared ACO state is initialized once; each query in turn generates and evaluates feasible candidate paths, updates the traffic state, and discourages the links it used, before the next query is processed in priority order.

$I(q)$ weights the query's declared priority ($w_p=0.65$) against the inverse of $d^*(q)$, the pure topological shortest distance ($w_d=0.35$; unreachable pairs get $d^* = 10^9$), so high-priority and shorter trips are routed first, freeing shared capacity for later queries. All 500 queries are sorted once by descending $I(q)$; pheromone is initialized *once* and persists across the whole run, so trails from earlier queries directly influence later ones. After each query is routed, the usage fraction ϕ_e of every link it uses is computed and pheromone on that link is discouraged in proportion to ϕ_e (strength $\theta=0.08$) – steering subsequent, lower-priority queries toward alternative, less-congested links. Unlike Method 1, the working traffic state is updated immediately after each query, so every query but the first is routed against a cost model reflecting all higher-priority congestion already assigned. Algorithm 3 summarizes Method 2, highlighting its four differences from Method 1: persistent pheromone, priority order, explicit discouragement, and immediate traffic update.

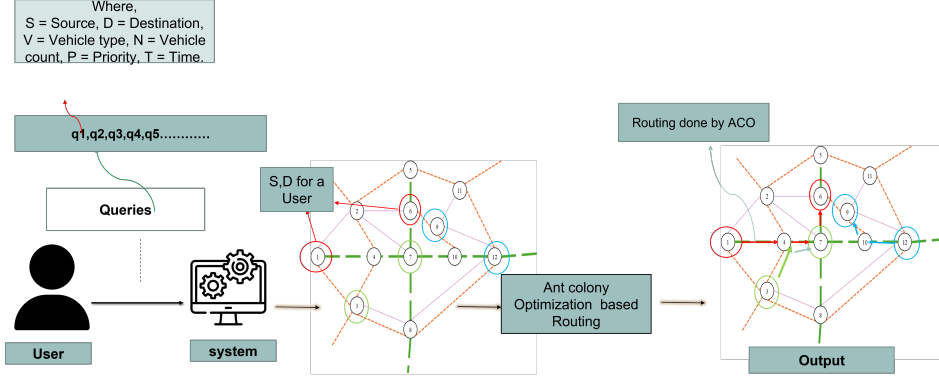


Figure III.11: Flowchart of the proposed congestion-aware ACO methodology, annotated with the road factor RF_{ij} , heuristic η_{ij} , transition rule P_{ij}^k , and pheromone update $\tau \leftarrow (1 - \rho)\tau + \Delta\tau$ used in this thesis.

Algorithm 3 Method 2 – Priority-Discouragement ACO

Require: dataset G , initial traffic state T_0 , queries $Q = \{q_1, \dots, q_{500}\}$

Ensure: route assignments and final traffic state T_{final}

- 1: **for each** $q_i \in Q$ **do**
 - 2: compute $I(q_i)$
 - 3: **end for**
 - 4: sort Q by descending $I(\cdot)$; initialize $\tau \leftarrow \tau_0$ once; $T_{\text{work}} \leftarrow T_0$
 - 5: **for each** $q_i \in Q$ in priority order **do**
 - 6: paths $\leftarrow \text{GENERATEPATHS}(q_i, \tau, T_{\text{work}}, G)$
 - 7: allocate q_i 's vehicles greedily across paths
 - 8: discourage τ on used links (Rule above); add assignment's vehicles to T_{work}
 - 9: **end for**
 - 10: $T_{\text{final}} \leftarrow T_{\text{work}}$; **return** assignments, T_{final}
-

3.6.3 Flowchart of the Proposed Method

Figure III.11 presents the complete pipeline – data/network setup, cost/heuristic modeling, improved ACO routing, and evaluation/output – annotated with the symbols defined above, consistent with Figure III.1.

3.7 Experimental Design and Validation Strategy

Three validation mechanisms address, respectively, internal validity, model correctness, and external comparability: a matched-conditions comparison of the two proposed methods, manual path evaluation of the cost model, and re-implementation-based comparison against related methods.

3.7.1 Matched-Conditions Comparison Design

Both methods run on the identical dataset, the same initial traffic state (time step 0), identical ACO parameters (Table III.8), and an identical seeded random generator (`random_seed= 42`). Method 2 is run on the same original input as Method 1, not on Method 1’s resulting traffic state, so the two methods are compared as alternative ways of routing the *same* demand against the *same* initial network state; every stochastic component is seeded, so the pipeline is fully reproducible.

3.7.2 Manual Path Evaluation

A manual path-evaluation utility accepts hand-specified alternative routes (`manual_node_paths.csv`), converts each node sequence to a link sequence, and scores it under the identical cost model (Equation III.8 and the cost-model Rule above), enabling the worked case-study comparison of Section 4.6.3 between an automated Method 1 route and manually constructed alternatives for the same query.

3.7.3 Common Evaluation Layer for Cross-Method Comparison

Because each re-implemented paper has a different native objective and output, a common evaluation layer appends an identical post-processing metrics stage to every implementation under five rules:

- Rule 1:** no solver logic is changed in any project – the layer only reads values already computed;
- Rule 2:** native output files stay byte-identical – the layer writes one *extra* file (`output/common_result.csv`);
- Rule 3:** a non-meaningful metric is recorded as N/A rather than invented;
- Rule 4:** runtime is measured around the solve call where separable, otherwise around the whole pipeline with a note; and
- Rule 5:** each paper’s own headline metric is preserved alongside the common metrics.

A self-contained header (`CommonResult.h`) is copied unchanged into every project; each project’s `main` is extended to emit one row per method into a fixed 18-column schema (method ID, dataset/query counts, success rate, cost, congestion, travel time, runtime, load-balance index, native headline metric, notes), merged across projects by `aggregate_common_results.py`. Metrics are defined in Section 3.8, compared methods listed in Section 4.3, and the resulting table analyzed in Section 4.7.

3.8 Evaluation Metrics

Rule — Total Cost, Average Congestion, Load Balance

Input: final traffic state of a method

Output: TC, AC, LB

System: $TC = \sum_a J(P_a, q_a) \times \text{vehicleCount}_a$ (lower is better)

$AC = \frac{1}{|E|} \sum_{e \in E} C_e$ (lower is better)

$LB = \frac{1}{1 + \sigma_u/(\bar{u} + \varepsilon)}, \quad u_e = L_e/\text{cap}_e$ (higher is better)

Total cost sums path cost (Section 3.5) weighted by vehicles carried, over every route assignment; average congestion is the mean per-link congestion coefficient across all 3,473 links; and load balance is a coefficient-of-variation score in $(0, 1]$ over per-link utilization u_e , closer to 1 when utilization is uniform across the network. Together these give a network-wide view beyond the aggregate travel-time/cost measures typically reported in Chapter II.

The common evaluation layer adds, for every method: **success rate** ($\text{success_count}/\text{num_queries}$); **runtime** (steady-clock milliseconds around the solve call); **Jain’s fairness index** [8],

$$\mathcal{J}(x_1, \dots, x_n) = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}, \quad (\text{III.18})$$

computed over per-link utilization ($x_i = u_e$), a second, independently defined load-balance measure; and **average travel time** where natively computable (N/A for the proposed framework, which has no explicit speed/time model). Because cost units and congestion definitions differ between papers, raw cost/congestion values are compared only *within* a project, never *across* projects; capabilities, success rates, runtimes, and fairness indices are compared across projects.

Three derived quantities recur in Chapter IV: relative improvement $\Delta M(\%) = (M_{\text{proposed}} - M_{\text{baseline}})/M_{\text{baseline}} \times 100$; average cost per served query $\overline{TC} = TC/\text{success_count}$; and the path-consolidation ratio $\text{PCR} = \#\text{assignment rows}/\#\text{served queries}$, used in Section 4.6.1 to compare output size without conflating it with queries served.

3.9 Novelty and Significance of the Proposed Method

No reviewed study integrates all four elements of the proposed framework: an explicit road-class access matrix; a PCU-weighted, capacity-derived congestion coefficient; a capacity-aware cost folded directly into the ACO heuristic in place of $1/d_{ij}$; and a priority-discouragement mechanism that orders queries by importance and actively steers later, lower-priority queries away from links used by earlier ones. This combination addresses network-scale, multi-query coordination – shown missing from all reviewed methods in Section 2.6 – on a realistic 1,168-node, 3,473-link network with 500 simultaneous heterogeneous queries, rather than the single-trip or small-scale settings used by prior work; every claimed element corresponds one-to-one to a mechanism defined above and exercised in Chapter IV. To the author’s knowledge, this is among the first studies to apply a congestion-aware ACO framework to a digitized central Dhaka network and the first to place it beside re-implemented related methods through a common evaluation layer. Chapter IV now presents the experimental studies conducted with this methodology.

CHAPTER IV

Experimental Studies

The proposed methodology was implemented as a modular C++17 application and evaluated through its experimental environment and software modules, parameter settings, baseline and comparative methods (including five re-implemented method families and the common evaluation layer), the procedure of experimentation, the experimental results and their analysis (query-success, manual case study, ablation, and scalability), the cross-method comparison, and a discussion against the stated objectives.

4.1 Experimental Environment

The framework is implemented entirely in standard C++17 (no external routing, GIS, or optimization libraries), built with Microsoft Visual Studio for x64 Release, with result aggregation in Python 3 (Table IV.1).

4.1.1 Project Structure and Source Code Modules

A single `main.cpp` driver runs the pipeline of Figure III.1: load/build the `Dataset`, run Method 1 then Method 2, compute metrics for both, save outputs, and run the manual evaluator if its input file is present. Table IV.2 maps each stage to its source module; the cost-model functions (`roadLoadPCU`, `congestionCoefficient`, `roadCapacityPCU`, `vehicleRoadCost`, `pathCost`, `pathCapacity`) directly implement the Rule of Section 3.5 and are called identically by both routing modules, ensuring both methods are compared under exactly the same cost definition.

4.1.2 Resource Budget

As a software-only project, costs are dominated by development time rather than physical components; no specialized hardware, paid dataset licenses, or paid tools were required (Table IV.3).

4.2 Parameter Settings

All experiments use the parameter values of Table III.8: 20 ants, 30 iterations, $\alpha = 1.0$, $\beta = 2.0$, $\rho = 0.20$, $\tau_0 = 1.0$, discouragement $\theta = 0.08$, priority weight $w_p = 0.65$, distance weight $w_d = 0.35$; capacity constants 1400/900/500 veh/hr/lane for highway/urban/local over a 60-second interval; and flow rates 18/12/7 by road class. Both

Table IV.1: Implementation environment used for developing and running the route optimization system.

Component	Detail
Programming language	C++17 (ISO/IEC 14882:2017)
Compiler / IDE	Microsoft Visual Studio (MSVC toolset), x64
Operating system	Microsoft Windows 11
Build configuration	Release (x64) for reported experiments
External libraries	None (C++ Standard Library only)
Input / output format	Comma-Separated Values (CSV)
Result aggregation	Python 3 (standard library only, for the common-layer aggregator)
Auxiliary analysis tools	Microsoft Excel (comparison spreadsheets and scatter-plot visualization)

routing methods share these parameters and a deterministic seed (`random_seed= 42`); the re-implemented comparison methods of Section 4.3 each use their own paper’s settings, seeded the same way.

4.3 Baseline and Comparative Methods

“Baseline” denotes the simple comparison run under identical conditions inside the same experiment: Independent ACO (Method 1, Section 3.6.1) shares the proposed method’s cost model, parameters, dataset, and seed, differing only in lacking the coordination layer of Figures III.9–III.10 – so it isolates exactly the proposed mechanism’s contribution.

Five related method families (Chapter II) were additionally re-implemented on the same Dhaka dataset (Table IV.4), and the common evaluation layer of Section 3.7.3 was appended to every project (identical `CommonResult.h`, steady-clock timers, no solver logic changed, byte-identical native outputs verified in Section 4.4) so all eleven method rows could be merged into one comparison table by `aggregate_common_results.py`.

4.4 Procedure of Experimentation

- (1) **Main run.** Method 1 runs first on the unmodified dataset (`method1_paths.csv`); Method 2 then runs on the *same* original input, not on Method 1’s resulting traffic state (`method2_paths.csv`); metrics for each method’s own final state go to `metrics.csv`.

Table IV.2: Source-code modules and the methodology section each implements.

Module (files)	Role	Implements
DatasetBuilder .h/.cpp	Loads CSVs, remaps IDs, builds adjacency and traffic-state matrix	Section 3.3
FlowGenerator .h/.cpp	Poisson-based inflow/outflow simulation	Section 3.4
QueryGenerator .h/.cpp (+ access-matrix/vehicle generators)	Weighted random query sampling	Section 3.3
CostModel.h/.cpp	Road load, congestion coefficient, capacity, road/path cost	Section 3.5
IndependentACO .h/.cpp	Method 1: per-query pheromone, ant search, Dijkstra fallback, allocation	Section 3.6.1
PriorityDiscouragementACO.h/.cpp	Method 2: importance sort, persistent pheromone, discouragement, traffic propagation	Section 3.6.2
ManualPathEvaluator .h/.cpp	Evaluates hand-specified node paths under the same cost model	Section 3.7.2
Metrics.h/.cpp, CommonResult.h	TC/AC/LB and the common-layer CSV output	Sections 3.8, 3.7.3

- (2) **Manual evaluation.** With `manual_node_paths.csv` present, the manual evaluator additionally produces `manual_evaluated_paths.csv` and `manual_metrics.csv`.
- (3) **Determinism check.** Each unmodified project, built in Release (x64), was executed twice; all six projects produced byte-identical outputs, confirming seeded determinism.
- (4) **Non-interference verification.** Comparing output folders before and after adding the common-evaluation-layer edits showed every native output file unchanged

Table IV.3: Approximate resource budget for the thesis project.

Category	Item	Approximate Cost
Software	Visual Studio Community Edition	Free (student license)
Software	Microsoft Excel (result visualization)	Institutionally licensed
Dataset	Road-network node/link data	Free (open mapping source)
Compute	Personal workstation (development and experiments)	Existing hardware, no incremental cost
Effort	Author development and analysis time	~26 weeks (Figure I.2)

Table IV.4: The six implementation projects included in the common cross-method comparison.

Project	Description	Method type
Proposed framework	Two multi-query routing methods on 500 OD queries with capacity limits: Independent ACO (baseline) and Priority-Discouragement ACO (proposal)	routing
Base paper: Wu <i>et al.</i> (2020) [3]	Improved ACO with road-condition factor plus PSO parameter tuning; 50 OD pairs routed independently; congestion-rate (CR) metrics	routing
Cheng (2023) [4]	Improved ACO with AHP-weighted comprehensive cost; one source–destination query in base (midday) and peak scenarios	routing (single OD)
Zhang <i>et al.</i> (2023) [5]	Combined dynamic route guidance and signal timing around a simulated accident; four internal methods A/B/C/D, where D is the paper’s proposal	routing + signal
Signal-timing optimization	Queue simulation on a 3-approach Dhaka sub-network with automatic green-time search; no OD routing	signal only
Physarum-inspired BLN (Akhand <i>et al.</i>) [12]	Physarum (slime-mould)-inspired bicycle-lane network design over 120 synthetic OD pairs; selects a connected low-cost sub-network rather than routing motor vehicles	network design

(byte-identical), and common-layer values matched the native metrics files exactly.

- (5) **Aggregation.** `aggregate_common_results.py` merges all six projects’ rows (eleven methods total) into `final_common_comparison.csv`.

Table IV.5 lists the files generated by the proposed framework’s pipeline: `method1_paths.csv` and `method2_paths.csv` carry one row per route assignment (`query_id`, `vehicle_type`, `vehicle_count`, `path_capacity`, `path_cost`, `edge_path`, `node_path`); `metrics.csv` carries TC/AC/LB per method; `common_result.csv` adds the 18-column common-layer row per method; and the three `manual_*` files carry the manual case study of Section 4.6.3.

Because a query’s demand may split across more than one candidate path, Method 1 produces 614 route-assignment rows while Method 2 produces 373; as the common-layer accounting in Section 4.5.3 shows, both methods serve the identical 351 of 500 queries, so this row-count gap reflects path consolidation, not a difference in queries served.

Table IV.5: Output files generated by the route optimization system.

Output file	Description	Rows
method1_paths.csv	Route assignments produced by Independent ACO	614
method2_paths.csv	Route assignments produced by Priority-Discouragement ACO	373
metrics.csv	Total cost, average congestion, and load balance per method	2
manual_node_paths.csv	Hand-specified alternative routes used as input to the manual evaluator	3
manual_evaluated_paths.csv	Cost, capacity, and status of each manually specified route	3
manual_metrics.csv	Metrics for each manually labeled route group	3
common_result.csv	Common-layer rows (18 columns) for both proposed methods	2

Table IV.6: Excerpt of method1_paths.csv for query 0 (node 1155 to node 355, private car).

Vehicles on this path	Path capacity	Path cost	Node path (excerpt)
1	1	5,554.8750	1155 → 64 → 50 → 789 → ... → 295 → 331 → 355
36	1	5,554.8750	Same lowest-cost path (overflow assignment)

4.5 Experimental Results

4.5.1 Result of Independent ACO

Applying Method 1 to the 500-query complete Dhaka dataset yields $TC_1 = 3.764944 \times 10^8$, $AC_1 = 2.174554$, and $LB_1 = 0.395797$ (Table IV.11), establishing the baseline against which priority ordering and discouragement are measured. Because every query is routed independently against the same static traffic state, this reflects what network-wide congestion would look like if all 500 trips were planned in isolation – a proxy for conventional per-trip route guidance.

Table IV.6 reproduces the two route rows generated for query 0 (source node 1155, destination node 355, private car, 37 vehicles, priority 3).

With only 1 PCU of spare capacity on this corridor’s bottleneck link, the greedy allocator assigns one vehicle within the residual capacity and then overflows the remaining 36 onto the same lowest-cost path rather than a costlier alternative – together accounting for the query’s full 37-vehicle demand. This query is also the manual case study of Section 4.6.3: the 5,554.875 figure here is a per-vehicle path cost, whereas the case study

Table IV.7: Common-layer measurements for the two proposed methods on the 500-query complete Dhaka dataset.

Common-layer metric	Independent ACO	Priority-Disc. ACO
Successful queries (of 500)	351	351
Success rate	0.702	0.702
Total cost	3.765×10^8	3.454×10^8
Average congestion	2.175	1.603
Solve runtime (ms)	6,517	13,311
Jain’s fairness index (all links)	0.300	0.388
Average travel time	N/A	N/A

reports the demand-weighted total of 287,157 for the whole assignment.

4.5.2 Result of Priority Discouragement ACO

Applying Method 2 to the identical dataset yields $TC_2 = 3.454061 \times 10^8$, $AC_2 = 1.603040$, and $LB_2 = 0.443135$ (Table IV.11). Every metric moves favorably relative to Method 1, indicating that priority ordering with pheromone discouragement spreads traffic more evenly without increasing cost. These gains are markedly larger than on the earlier sparse extract (Section 4.7.2), since a denser network offers more genuinely distinct corridors for discouragement to exploit.

4.5.3 Common-Layer Results for the Proposed Methods

Table IV.7 reports the common-layer row for each proposed method.

Both methods succeed on *exactly the same* 351 of 500 queries – the 149 failures are identical for both, localizing them in the dataset rather than either algorithm (Section 4.6.1). Method 2’s 13.3-second solve time (about 27 ms/query) is only $2.04\times$ the baseline’s 6.5 seconds, keeping the coordination overhead well within real-time-capable territory. Jain’s index improves from 0.300 to 0.388 (29.1%), independently corroborating the load-balance gain under a second, differently defined fairness measure. Average travel time is honestly N/A for both, since the composite cost model has no explicit speed/time component.

Table IV.8: Query success rate by vehicle class over the 500-query set (identical for both proposed methods).

Vehicle class	Queries	Served	Success rate
Motorcycle	150	127	0.847
CNG	112	96	0.857
Private car	131	109	0.832
Bus	44	18	0.409
Rickshaw / bicycle	63	1	0.016
All classes	500	351	0.702

4.6 Experimental Analysis

4.6.1 Route Consolidation and Query-Success Analysis

Raw row counts alone (614 vs. 373) could only suggest that *at least* 127 queries produced no output under Method 2, raising a concern that discouragement might starve lower-priority queries. The common layer’s per-query accounting resolves this: both methods succeed on exactly the same 351 queries and fail on exactly the same 149. Because Method 1 routes every query independently with a guaranteed Dijkstra fallback, a failing query must be structurally infeasible (e.g., a rickshaw query needing arterial roads, or a source with out-degree zero, Section 3.4) – so the identical failure set under Method 2 shows discouragement causes no additional failures; the 149 failures (29.8%) are a property of the query set, not either algorithm.

The row-count gap instead reflects path consolidation (Equation III.19): over the same 351 served queries, Method 1 produces $PCR = 1.749$ paths/query (153 queries split across multiple paths), while Method 2 produces $PCR = 1.063$ (15 queries split). Because Method 2 routes against accumulated higher-priority traffic and discouraged pheromone on heavily used links, its search is steered toward comparatively fresh, higher-capacity corridors that can absorb a whole fleet on one path, whereas Method 1’s repeated selection of the same attractive corridors saturates them and forces splitting.

4.6.2 Query Success by Vehicle Class and Priority

Table IV.8 and Figure IV.1 break the 351 served/149 failed queries down by vehicle class (identical for both methods).

The broadly permitted classes (motorcycle, CNG, car) succeed at 83–86%, while rickshaw/bicycle succeeds on only 1.6% and bus on 40.9% – a direct consequence of the access matrix (Table III.6): rickshaws are barred from arterial roads, which form the

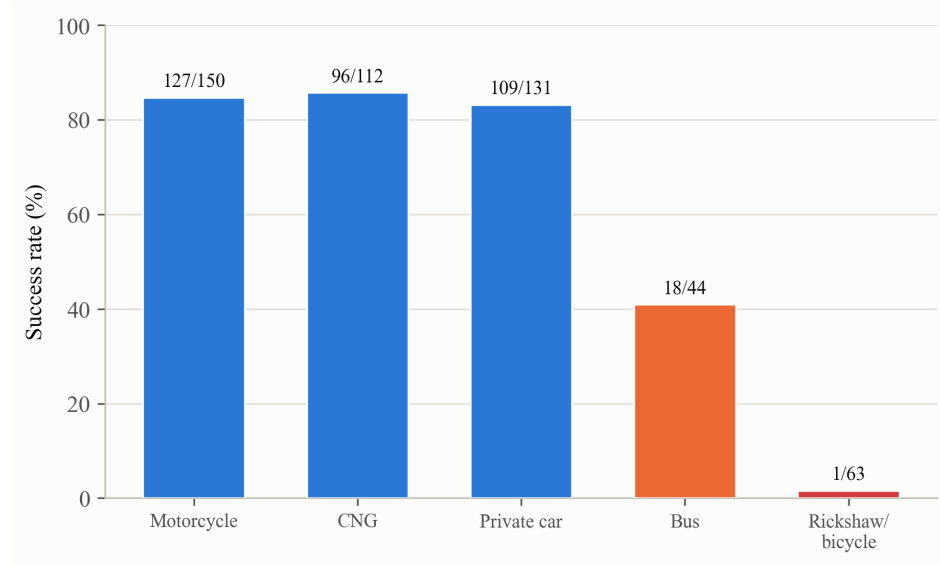


Figure IV.1: Query success rate by vehicle class (identical for both proposed methods). The two access-restricted classes – bus (no local roads) and rickshaw/bicycle (no arterial roads) – account for almost all of the failures.

Table IV.9: Manual path-evaluation case study for query 0 (37-vehicle private-car trip, node 1155 to node 355).

Route	Total Cost	Average Congestion	Load Balance
Independent ACO (original)	287,157.00	0.201103	0.517552
Manual modification 1	409,428.13	0.201074	0.517572
Manual modification 2	307,840.00	0.198965	0.517343

largest share of the network and most of its long-range connectivity, and buses are barred from local roads that often connect to the final destination. Of the 149 failures, 22 additionally have an out-degree-zero source node. By priority level, success is far more uniform (0.676–0.774 across priorities 1–5), confirming that priority governs only processing *order* (Equation III.12), not query feasibility, which is fixed by source, destination, and access constraints.

4.6.3 Manual Route Modification Case Study

Query 0 (37-vehicle private-car trip, node 1155 to node 355) was evaluated on three routes: the Independent ACO route and two manually constructed diversions with extra intersections inserted (Table IV.9).

The automatically generated route achieves the lowest cost, with the two manual diversions costing 42.6% and 7.2% more; network-wide congestion and load balance barely move, since a single 37-vehicle trip perturbs only a handful of the 3,473 links. This

Table IV.10: Solve-time scaling of the proposed methods from the sparse extract to the complete Dhaka dataset (500 queries in both cases).

Quantity	Sparse extract	Complete dataset	Ratio
Nodes	488	1,168	$2.39\times$
Directed links	1,435	3,473	$2.42\times$
Step budget maxSteps	981	2,341	$2.39\times$
Independent ACO runtime (ms)	3,731	6,517	$1.75\times$
PD-ACO runtime (ms)	8,397	13,311	$1.59\times$
PD-ACO per query (ms)	16.8	26.6	$1.59\times$

confirms the ACO search finds a genuinely lower-cost route than reasonable hand-built alternatives for the same origin–destination pair – a human-verifiable check on the cost model, not a claim of global single-trip optimality, since the ACO route minimizes the composite multi-query objective rather than this one trip alone.

4.6.4 Contribution of the Coordination Mechanism

Method 1 and Method 2 share the identical cost model, path-construction procedure, parameters, dataset, and seed; Method 2 differs only in priority ordering, persistent pheromone with discouragement, and per-query traffic propagation. This matched-conditions design is effectively a two-arm ablation of the whole coordination layer, attributing the measured 8.26% cost, 26.28% congestion, 11.96% CV-load-balance, and 29.13% Jain-index improvements to it collectively, at $2.04\times$ runtime. The largest gains fall exactly on the distribution-oriented metrics that discouragement directly targets, while the cost gain is smaller and indirect. Ablating the three components individually was not instrumented here and is left for future work (Section 7.4).

4.6.5 Runtime and Scalability Analysis

Table IV.10 compares solve times on the sparse extract versus the complete dataset.

Although network size grows $\approx 2.4\times$, solve time grows only $1.6\text{--}1.8\times$: average out-degree stays roughly constant (≈ 2.97), so per-step work does not grow with the network, and maxSteps is only a loose upper bound most paths never approach. At 26.6 ms/query on the complete network, a full 500-query re-plan stays comfortably interactive, and since each query’s ant construction is independent, the single-threaded implementation leaves an unexploited parallelism factor for production deployment (Section 7.4).

4.7 Comparative Analysis

Table IV.11 reports the full contents of `metrics.csv`.

Table IV.11: Comparison of Independent ACO and Priority-Discouragement ACO on the 500-query complete Dhaka dataset (full contents of `metrics.csv`).

Method	Total Cost (TC)	Average Congestion (AC)	Load Balance (LB)
Independent ACO	3.764944×10^8	2.174554	0.395797
Priority Discouragement ACO	3.454061×10^8	1.603040	0.443135
Improvement	−8.26%	−26.28%	+11.96%

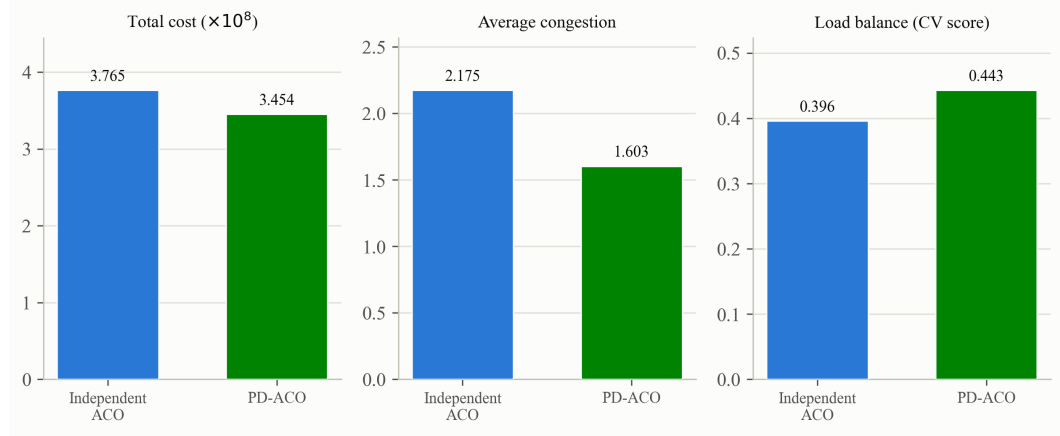


Figure IV.2: Native-metric comparison of Independent ACO and Priority-Discouragement ACO on the complete Dhaka dataset: total cost, average congestion, and coefficient-of-variation load balance.

Priority-Discouragement ACO cuts total cost by 8.26% (Section 3.5’s congestion penalty term $(1 + C_e)$ falls as traffic is redistributed, even though discouragement modifies pheromone rather than cost directly), cuts average congestion by 26.28% – the largest relative gain, computed as a mean over *all* 3,473 links, so it reflects reduced concentration on the most-loaded links, not just marginally cheaper individual paths – and raises load balance by 11.96% (a lower coefficient of variation of per-link utilization), corroborated by Jain’s index moving from 0.300 to 0.388 (29.1%). Figure IV.4 gives a complementary log-scale view of the same three native metrics, showing at a glance how far apart the two methods’ total cost, average congestion, and load balance sit relative to one another.

4.7.1 Cross-Method Comparison Through the Common Evaluation Layer

Table IV.12 condenses the aggregated common-evaluation-layer comparison across all eleven method rows from the six implementation projects.

Figures IV.5–IV.7 plot the directly comparable metrics (green: proposed main method; blue: internal baseline; gray: re-implemented comparisons; N/A rows omitted except in the log-scale runtime chart).

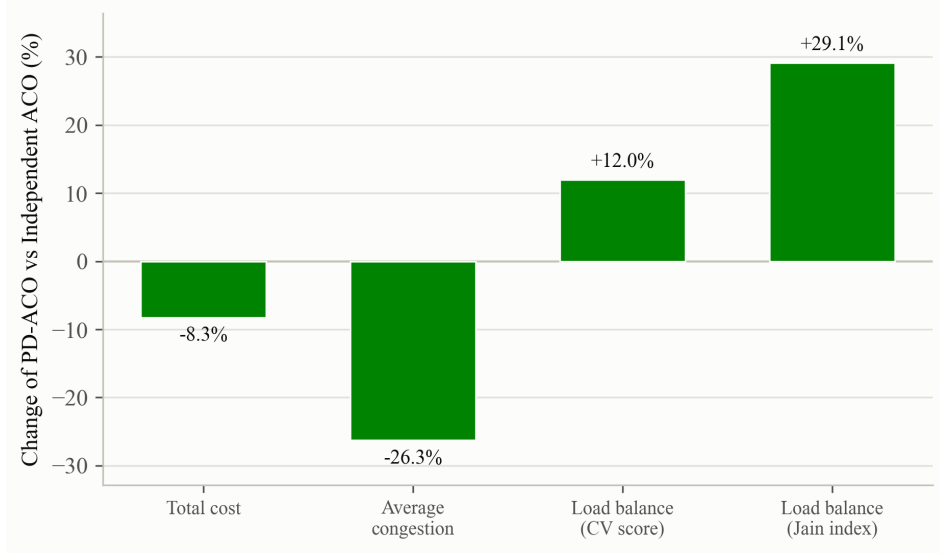


Figure IV.3: Relative change of Priority-Discouragement ACO over the Independent ACO baseline on the complete Dhaka dataset (negative is favourable for cost and congestion; positive is favourable for the two load-balance measures). Runtime rises by a factor of 2.04.

Four findings follow. **Capability:** the proposed method is the only approach that jointly optimizes 500 simultaneous, capacity-constrained queries over the whole network; Cheng optimizes one vehicle, the base paper routes 50 pairs independently, Zhang reroutes around one accident (its network-wide utilization is essentially unchanged at 0.563 across all four internal methods, confirming local rather than network-level redistribution), the signal-timing project performs no routing, and Physarum designs bicycle lanes rather than motor-vehicle routes. **Fair win:** within the only truly controlled pairing, Priority-Discouragement ACO beats Independent ACO on every figure (8.3% lower cost, 26% lower congestion, 29% higher Jain balance). **Success rate in context:** the 70.2% rate reflects dataset infeasibility, not algorithmic weakness (Section 4.6.1); the closest comparable method (base-paper ACO) reaches 68% on a query set ten times smaller, while the 100% figures for Cheng’s single query and Physarum’s synthetic demand are not meaningfully comparable to a 500-query task. **Speed:** at ≈ 27 ms/query, the proposed method matches the other methods’ millisecond-per-decision pace while solving a much larger joint problem.

4.7.2 Effect of Dataset Completeness

Table IV.13 contrasts the proposed methods on the earlier sparse extract (488 nodes, 1,435 links) versus the complete dataset (1,168 nodes, 3,473 links, Figure IV.8).

On the complete network, the coordination layer’s advantage is substantially larger (cost gap 1.06% $\rightarrow$ 8.26%, congestion gap 11.34% $\rightarrow$ 26.28%, Jain gap 17.0% $\rightarrow$ 29.13%): a denser network gives discouragement more genuinely distinct corridors to redistribute

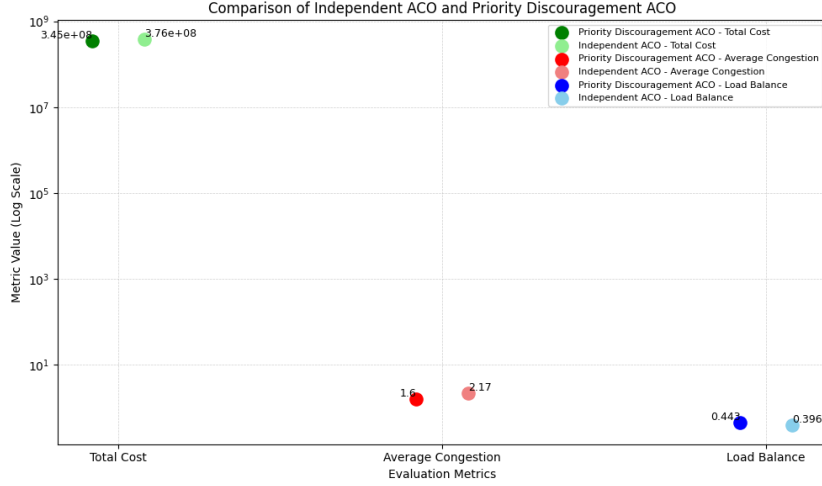


Figure IV.4: Log-scale comparison of total cost, average congestion, and load balance for Independent ACO and Priority-Discouragement ACO on the complete Dhaka dataset, with the exact values of Table IV.11 annotated on each point.

traffic onto. Raw magnitudes are not comparable across datasets (e.g., the baseline’s absolute congestion is *lower* on the complete network, 2.17 vs. 4.44, simply because demand spreads over more than twice as many links); the honest reading is the within-dataset gap between the two proposed methods.

4.7.3 Within-Family Analyses of the Re-implemented Methods

Since cost and congestion units are not comparable across papers, each re-implementation is checked against its own source paper’s qualitative behavior. **Zhang *et al.*:** average delay falls from 313.98 s (no optimization) to 307.77 s (route guidance) to 269.77 s (signal timing) to 264.19 s (combined, 15.9% below baseline), confirming the paper’s finding that combining routing and signal control beats either alone (Figure IV.11). **Cheng:** comprehensive path cost rises modestly from 2.0537 (midday) to 2.0811 (peak), and mean path congestion from 0.272 to 0.396, correctly registering heavier peak traffic (Figure IV.12). **Physarum BLN:** connects all 120 synthetic demands and selects a 391.5 km lane network in ≈ 7 ms, with routing-specific fields honestly marked N/A since a network-design objective has no vehicular congestion notion.

4.8 Discussion of Findings

Three trends stand out. First, the gain is uneven across metrics – congestion (26.28%) and both load-balance measures (11.96%, 29.13%) improve more than total cost (8.26%) – because discouragement (Equation III.14) directly targets pheromone on heavily used links, i.e., traffic *distribution*, and the cost gain follows only indirectly through reduced congestion penalties, more strongly on the complete network than the sparse extract.

Table IV.12: Cross-method comparison of all eleven method rows through the common evaluation layer (complete-dataset run of 4 July 2026). Cost and congestion units differ per paper and must not be compared across projects; capabilities, success rates, runtimes, and fairness indices are comparable.

Method	Type	Nodes/ links	Success rate	Runtime (ms)	Jain in- dex	Native headline metric
Proposed Priority-Disc. ACO (main)	routing	1168/ 3473	0.702 (351/500)	13,311	0.388	total cost 3.454×10^8
Proposed Independent ACO (baseline)	routing	1168/ 3473	0.702 (351/500)	6,517	0.300	total cost 3.765×10^8
Base paper improved ACO + PSO (Wu 2020)	routing	1168/ 3477	0.680 (34/50)	6,903	0.709 ^a	$CR_{opt} = 0.106$
Cheng 2023 improved ACO (base scenario)	routing (1 OD)	1061/ 3084	1.0 (1/1)	42	N/A	compr. cost 2.0537
Cheng 2023 improved ACO (peak, Eq. 9)	routing (1 OD)	1061/ 3084	1.0 (1/1)	69	N/A	compr. cost 2.0811
Zhang 2023 A: no optimization (baseline)	baseline	1168/ 7124	0.967 (58/60)	620 ^b	0.543	avg. delay 313.98 s
Zhang 2023 B: route guidance	routing	1168/ 7124	0.983 (59/60)	620 ^b	0.543	avg. delay 307.77 s
Zhang 2023 C: signal timing	signal only	1168/ 7124	0.967 (58/60)	620 ^b	0.543	avg. delay 269.77 s
Zhang 2023 D: combined (Zhang’s main)	routing + signal	1168/ 7124	0.983 (59/60)	620 ^b	0.543	avg. delay 264.19 s
Signal-timing auto-optimized plan	signal only	1168/ 3564	N/A	20,254	N/A	weighted queue 115.0 m
Physarum-inspired BLN (Akhand <i>et al.</i> 2021)	network design	1098/ 3366	1.0 (120/120)	7	0.181 ^c	BLN length 391.5 km

^a Jain index computed over used-link route counts only (a different definition, not comparable to the proposed method’s all-link Jain index). ^b Shared pipeline time for all four Zhang methods; per-method time is not separable by design. ^c Jain index computed over selected bicycle-lane usage, a network-design objective rather than vehicular load; not comparable to the routing rows. Node/link counts differ per row because each implementation applies its own network cleaning; Zhang, for instance, synthesizes 7,124 one-way directed links from the shared bidirectional base.

Second, the common evaluation layer converted an apparent caveat (Method 2’s smaller output file) into a dataset-level finding (identical 351-query success for both methods, Section 4.6.1) without touching solver logic – confirming that, under common metrics, the proposed framework uniquely coordinates network-scale simultaneous demand at a competitive runtime. Third, the manual case study (Section 4.6.3) shows hand-built alternatives can cost over 40% more than the ACO route, underscoring that cost-model and search quality, not mere road availability, determine whether congestion-aware routing helps.

Objectives achieved. The Dhaka network and fleet were modeled as specified (Objective 1, Sections 3.3, 3.3); the cost function was formulated and used as the ACO heuristic (Objective 2, Section 3.5); both strategies were implemented and applied to the identical dataset under identical parameters (Objective 3, Sections 3.6.1, 3.6.2); they were compared on TC/AC/LB with the cost model verified by the manual case study (Objective 4, Sections 4.7, 4.6.3); and five related methods were re-implemented and

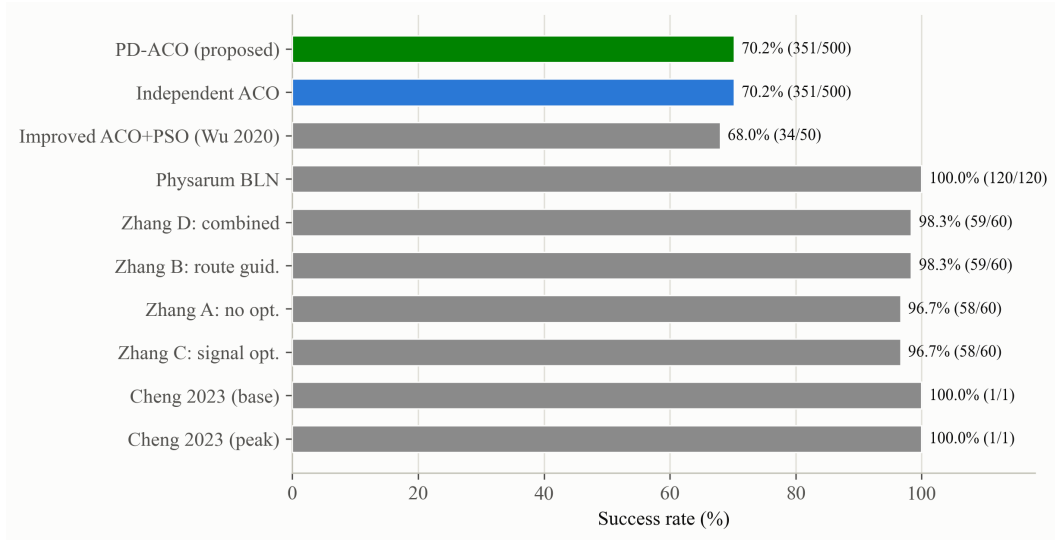


Figure IV.5: Success-rate comparison of the method rows reported in Table IV.12 (routing and network-design rows; signal-only rows omitted as success rate is N/A by design).

compared through the common layer without modifying any algorithm (Objective 5, Sections 4.3, 4.7.1). All five objectives, and the overall aim of Section 1.4, are considered achieved within the scope of Section 1.7.

4.9 Chapter Summary

Evaluated on the complete 1,168-node, 3,473-link Dhaka network with 500 heterogeneous queries, Priority-Discouragement ACO reduced total cost by 8.26%, average congestion by 26.28%, CV-based load balance by 11.96%, and Jain’s index by 29.13% over Independent ACO, at $2.04\times$ runtime. The common evaluation layer verified byte-identical native outputs, established that both methods serve exactly the same 351 queries (locating the 149 failures in the dataset, not the algorithms), and placed the proposed method alongside five re-implemented families, where it uniquely optimizes network-scale simultaneous demand. Dataset completeness widens the proposed method’s advantage, and the manual case study confirms the ACO route outperforms hand-built alternatives. Chapter V now turns to the engineering, ethical, and professional considerations of deploying a routing system of this kind.

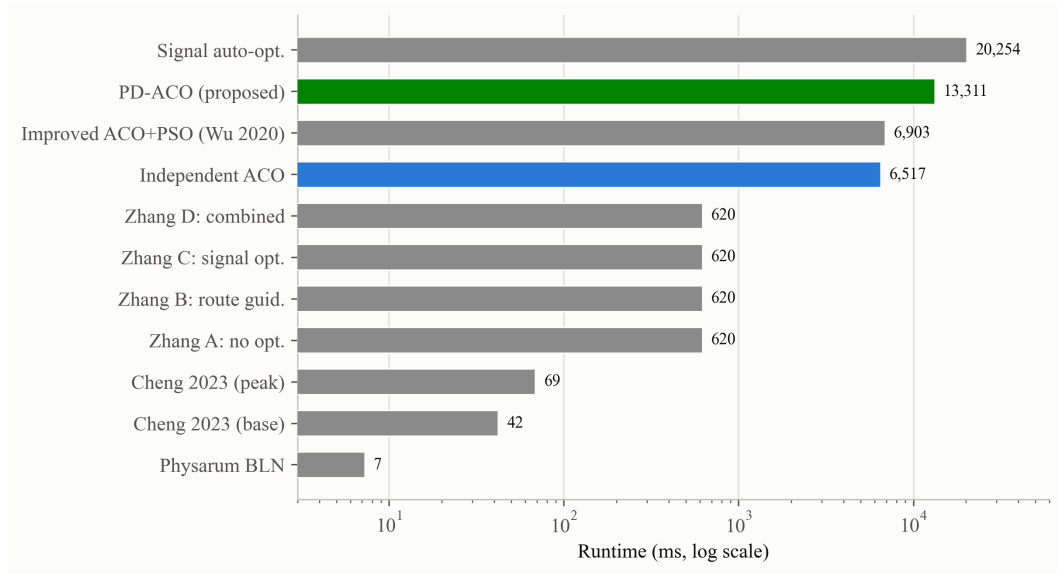


Figure IV.6: Runtime comparison (log scale) of the eleven method rows reported in Table IV.12. Zhang rows share one pipeline time; the base paper’s runtime includes PSO parameter tuning.

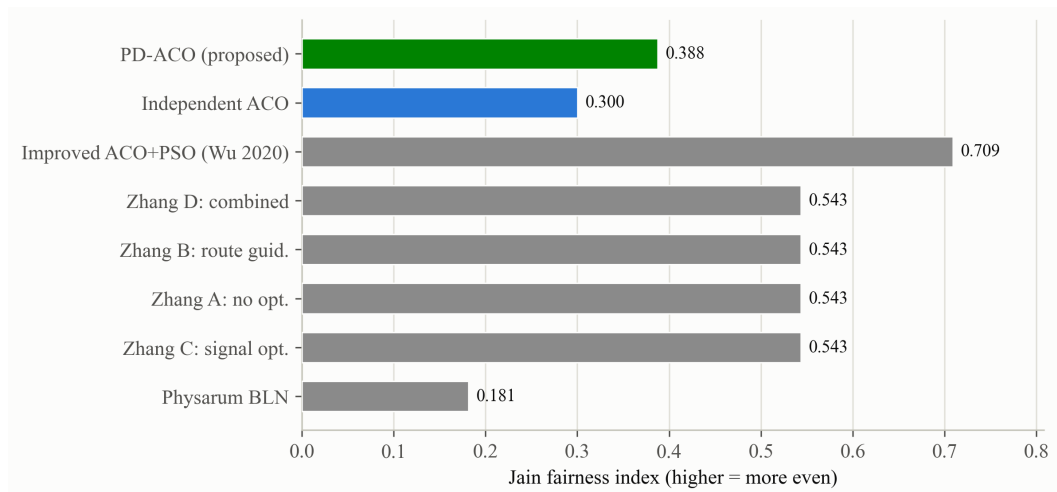


Figure IV.7: Jain fairness index of the rows where per-link load balance is defined. Definitions of the underlying load vector differ per family (see table footnotes), so within-family comparison is the intended reading.

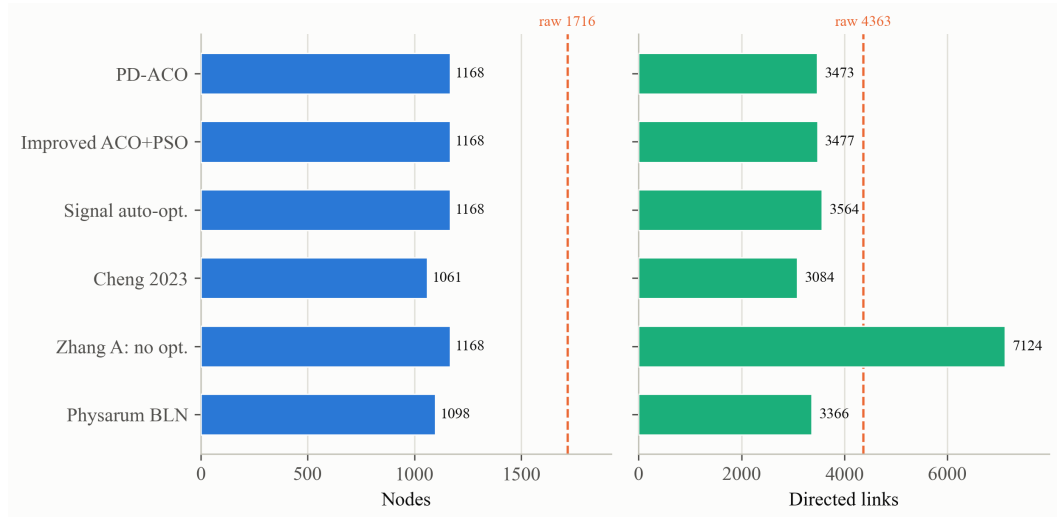


Figure IV.8: Node and directed-link coverage of each project's cleaned working network, compared with the raw collection (dashed line). Each implementation applies its own cleaning rule, so the retained subgraph differs per project.

Table IV.13: Proposed methods on the earlier sparse extract versus the complete Dhaka dataset. Because the two datasets have different link sets, the meaningful reading is the direction of change and the gap between the two methods, not the raw magnitudes.

Quantity	Sparse extract	Complete dataset	Direction
Nodes / links	488 / 1,435	1,168 / 3,473	—
Served queries (of 500)	329	351	higher
Success rate	0.658	0.702	higher
PD-ACO vs IACO: total cost gap	−1.06%	−8.26%	wider
PD-ACO vs IACO: congestion gap	−11.34%	−26.28%	wider
PD-ACO vs IACO: Jain-index gap	+17.0%	+29.13%	wider

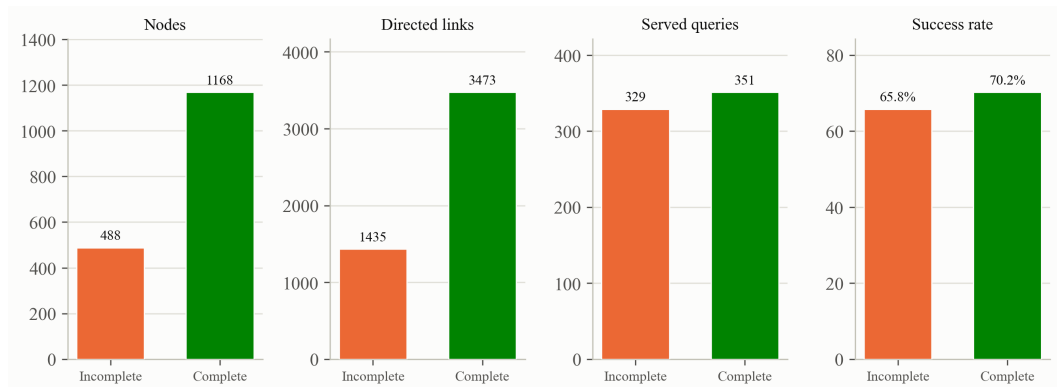


Figure IV.9: Effect of dataset completeness on the proposed Priority-Discouragement ACO: network size, served queries, and success rate on the sparse extract versus the complete Dhaka dataset.

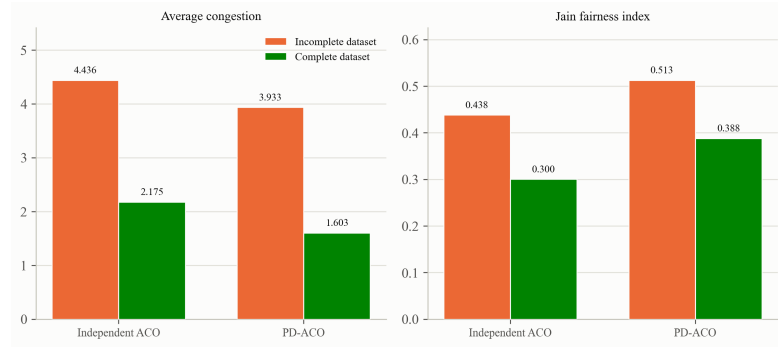


Figure IV.10: Average congestion and Jain fairness index for both proposed methods on the sparse extract versus the complete dataset. The link sets differ between datasets, so the comparison documents the dataset effect rather than a like-for-like magnitude change.

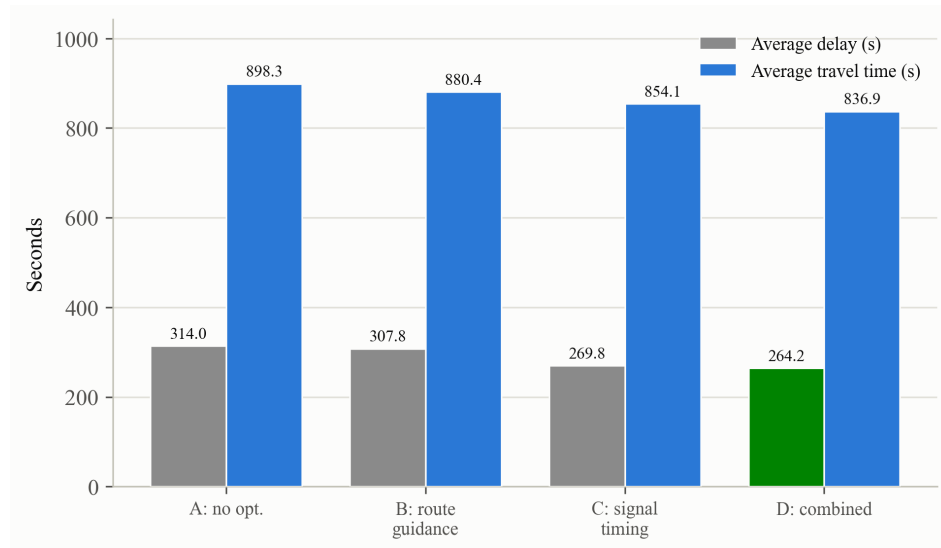


Figure IV.11: Within-family comparison of the four Zhang *et al.* (2023) methods on the Dhaka accident scenario: native average delay and average completed-trip travel time. Lower is better; the combined method D is best.

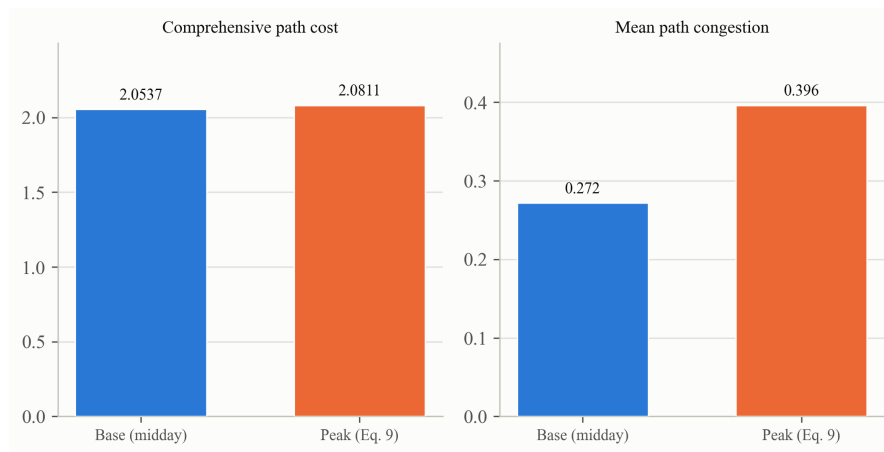


Figure IV.12: Cheng (2023) improved-ACO re-implementation: comprehensive path cost and mean path congestion for the base (midday) and peak scenarios on the same single origin–destination query.

CHAPTER V

Engineering, Ethical, and Professional Considerations

Although this thesis is primarily a technical study, a route-optimization framework intended for eventual deployment on a real public road network cannot be judged on algorithmic performance alone; it must also be examined for engineering soundness, intellectual property, ethics, privacy and security, safety, legality, and societal and environmental consequences, each considered here for the proposed congestion-aware ACO framework specifically.

5.1 Introduction

These considerations matter because the proposed framework models real infrastructure (the Dhaka road network) and safety-critical activity (motor-vehicle movement), so its design choices carry consequences beyond simulation accuracy.

Engineering-wise, the system uses a modular architecture (Section 4.1.1) in which the cost model, dataset builder, and routing algorithms communicate through well-defined CSV interfaces and a shared `Dataset` structure, so any component can be modified independently. Reliability is supported by the Dijkstra fallback (Section 3.6), which guarantees a feasible path whenever one exists, and by the deterministic-seeding and byte-level verification protocol of Section 4.4. Scalability is currently bounded by both methods being single-threaded, in-memory computations over the full network per query; a city-wide deployment would likely need parallelized ant construction or hierarchical routing (Section 7.4).

5.2 Intellectual Property Considerations

The road-network dataset was constructed by the author from openly available mapping information (Section 3.3) and is an original compilation, not a redistribution of proprietary data. The cost model (Section 3.5) and the priority-discouragement mechanism (Section 3.6.2) are original work developed under supervision; ideas drawn from prior work – the road-condition-factor concept [3] and the general ACO formulation [2] – are cited throughout Chapters II and III and extended rather than reproduced. The re-implemented comparison methods (Section 4.3) were written independently from the published papers’ descriptions and are clearly labeled as re-implementations. Any

future commercialization would additionally need to verify the licensing terms of the underlying mapping source.

5.3 Ethical Considerations

A system that discourages reuse of heavily loaded links, as Method 2 does, raises a fairness question: could some travelers be systematically routed onto longer paths so others' journeys shorten? Here, discouragement (Section 3.6.2) scales only with how heavily a link was already used and does not deny access beyond the access-matrix restrictions of Table III.6 (general Dhaka traffic regulation, not user-specific); a real deployment should nonetheless monitor whether lower-priority queries suffer disproportionately over repeated use and should set query "priority" (Equation III.12) through transparent policy. Because routing follows an explicit, published cost formula (Equation III.7) rather than an opaque learned model, any specific recommendation can, in principle, be explained to an affected user.

This thesis reports both favorable results (Section 4.7) and limitations (Section 7.3), including reliance on synthetic traffic and the 149 infeasible queries; all parameters and statistics are taken directly from the checked-in code and data, with a fixed seed (`random_seed= 42`) for reproducibility. The common evaluation layer, deliberately post-processing only with byte-level verification (Section 4.4), guards against accidental manipulation of comparison results, and re-implementations are explicitly disclosed as such rather than validated reproductions of the original authors' software.

5.4 Data Privacy and Security Considerations

Every input here is public infrastructure data or synthetically generated traffic/queries; no personal or identifying information is processed, so the reported experiments raise no direct privacy risk. A live deployment, however, would ingest real trip requests (origin, destination, vehicle type, timestamp) – personal location data requiring: (i) data minimization; (ii) anonymized/aggregated trip and link-level records so individual movements cannot be reconstructed; (iii) encryption and role-based access control; (iv) enforced retention limits; and (v) authenticated, plausibility-checked live data feeds, since a routing system that trusts unverified congestion reports is exposed to data-poisoning attacks that divert traffic for private benefit.

5.5 Safety Considerations

The vehicle-class access matrix (Table III.6) already encodes a basic safety constraint – preventing buses on narrow local roads and rickshaws/bicycles on high-speed arterial

roads – but the cost model does not yet represent road-surface condition, lighting, pedestrian density, or accident-prone locations; the road-condition-factor concept of Section 2.4.2 could be extended with such terms in future work. As an offline simulation study, no recommendation here has guided a real vehicle; a field trial would require staged evaluation against professional traffic-engineering judgment before influencing driver behavior.

5.6 Legal Considerations

The access matrix is designed to be consistent with general, publicly known Dhaka vehicle-access regulations, but this thesis does not claim exhaustive verification against the current legal traffic code for every road segment; public deployment would need validation against applicable traffic authorities’ regulations. A system ingesting real trip data would additionally need to comply with data-protection regulations (Section 5.4) and, for field trials with real drivers, institutional human-subjects requirements.

5.7 Societal, Health, and Cultural Impact

Dhaka’s congestion imposes social costs beyond lost time – stress, reduced productivity, delayed emergency access, and disproportionate impact on commuters relying on congestion-sensitive shared or non-motorized modes (bus, CNG, rickshaw) – plus direct health effects from stop-and-go exhaust exposure. A congestion-aware framework that can prioritize emergency-vehicle routing (Section 1.6) can improve outcomes for these groups if deployed responsibly. Culturally, the framework explicitly models rickshaws and CNGs that mainstream navigation systems typically ignore, respecting rather than overriding Dhaka’s actual mobility mix. Because it currently relies on synthetic rather than live data (Section 1.7), its societal benefit here is evidence of algorithmic potential at simulation level, not proven field impact.

5.8 Environmental and Sustainability Considerations

Congestion is strongly associated with higher fuel consumption and emissions per distance, since stop-and-go idling is markedly less efficient than steady free-flow travel. The 26.28% average congestion reduction and 11.96% load-balance improvement (Section 4.7) suggest a plausible pathway to reduced fuel use and emissions if translated into a live system, though a rigorous estimate would require an explicit emissions model, left for future work. The method’s own computational footprint is modest (8.4 s of single-threaded computation for 500 queries, Section 4.5.3), so it does not trade transport emissions for significant computing-energy cost; more evenly distributed load also reduces noise and air-quality concentration on the small set of corridors that otherwise

absorb a disproportionate traffic share, complementing the green-mobility goals of the related bicycle-lane study [12].

5.9 Summary

The framework is modular and reliable but currently scalability-limited at city-wide scale; built on an original dataset and algorithm with independently written comparison re-implementations; raises fairness and transparency questions partially, not fully, addressed by the current design; processes no personal data now but would require minimization, anonymization, encryption, and integrity safeguards if deployed live; incorporates a basic access-based safety constraint without detailed road-safety modeling; is consistent with, but not exhaustively validated against, Dhaka traffic law; and offers plausible societal, health, and environmental benefits demonstrated here at simulation level. These considerations inform the limitations of Section 7.3 and the future work of Section 7.4.

CHAPTER VI

Complex Engineering Problems and Activities

The thesis maps to recognized Complex Engineering Problem (CEP) and Complex Engineering Activity (CEA) attributes under the Washington-Accord-aligned framework used to assess undergraduate engineering theses.

6.1 Introduction

Designing a congestion-aware routing framework for a real, large, heterogeneous network such as Dhaka's requires integrating graph modeling, stochastic traffic estimation, multi-criteria cost design, metaheuristic search, and quantitative evaluation into one coherent system, while managing genuine trade-offs (search quality versus computation time; individual-query optimality versus network-wide fairness).

6.2 Complex Engineering Problems Associated with the Project

Table VI.1 maps the standard Complex Engineering Problem (P1–P7) attributes to this thesis.

Table VI.1: Mapping of the thesis to Complex Engineering Problem (CEP) attributes.

Attribute	Description	Relevance to this thesis
P1	Depth of knowledge required	Requires knowledge of graph theory and shortest-path algorithms, stochastic processes (Poisson-based flow modeling), metaheuristic optimization (Ant Colony Optimization), transportation-engineering concepts (Passenger Car Units, road capacity, level-of-service-style congestion coefficients), and software engineering for a modular C++ simulation system.

continued on next page

Table VI.1 continued from previous page

Attribute	Description	Relevance to this thesis
P2	Conflicting requirements	Minimizing individual route cost can conflict with network-wide load balance (the shortest available path for every query, chosen independently, tends to concentrate load on the same corridors); the thesis explicitly manages this trade-off through the priority-discouragement mechanism of Section 3.6.2.
P3	Depth of analysis required	Requires deriving and combining several interacting sub-models (congestion coefficient, road capacity, access-permission cost, path capacity) into a single cost function, and requires statistical interpretation of the resulting total cost, average congestion, and load-balance metrics across 500 queries and two routing strategies.
P4	Familiarity of issues	The problem combines a well-studied algorithmic core (ACO, Dijkstra) with issues that are less standard and specific to this application: heterogeneous vehicle-class access restrictions, PCU-based mixed-traffic load, and priority-ordered, congestion-propagating multi-query routing on a real Dhaka network.
P5	Extent of applicable codes	The vehicle-class access matrix (Table III.6) is designed to be consistent with general Dhaka road-class and vehicle-access practice, though formal traffic-engineering codes and standards were not exhaustively applied (Section 5.6).

continued on next page

Table VI.1 continued from previous page

Attribute	Description	Relevance to this thesis
P6	Extent of stakeholder involvement	The primary stakeholders are road users of different vehicle classes (whose access and priority are explicitly modeled), traffic-management authorities (who could use the load-balance metric to target interventions), and the academic supervisor, whose guidance shaped the problem formulation and cost model (Section 1.9); broader public stakeholder consultation was outside the scope of this undergraduate thesis.
P7	Interdependence	The system is highly interdependent: the traffic-flow model feeds the cost model, the cost model feeds the ACO heuristic, the ACO output feeds the traffic update in Method 2, and the updated traffic state feeds back into the cost model for subsequent queries (Section 3.6.2), forming a closed loop whose overall behavior cannot be understood by examining any single module in isolation.

This depth and interdependence are sharpened by scale: optimizing 500 heterogeneous, capacity/access-constrained queries jointly over a 1,168-node, 3,473-link graph is not a matter of running a textbook shortest-path routine, since each query interacts with every other through the shared, evolving traffic state and feasibility depends on the interaction between vehicle class, access matrix, and local road-type composition (Section 3.4) – the near-total exclusion of rickshaw trips from arterial-heavy corridors (Section 4.6.2) demonstrates this concretely. Doubling the network size roughly doubled the per-query solve time ($6.5 \rightarrow 13.3$ s for the full run), the scale-dependent behavior that motivated the 20-ant/30-iteration parameter choice of Section 3.6.

6.3 Complex Engineering Activities Associated with the Project

Table VI.2 maps the standard Complex Engineering Activity (A1–A7) attributes to this thesis.

Table VI.2: Mapping of the thesis to Complex Engineering Activity (CEA) attributes.

Attribute	Description	Relevance to this thesis
A1	Range of resources	Involves open digital mapping data, a custom-built C++17 simulation framework, five separately built comparison projects with a shared common evaluation layer and a Python aggregation script, standard development tools (Visual Studio), and analysis software (Microsoft Excel) integrated into a single reproducible experimental pipeline (Section 4.1).
A2	Level of interaction	Requires coordinating multiple software modules (dataset builder, flow generator, cost model, two ACO routing modules, manual evaluator, metrics calculator) that must interact through a consistent shared data representation (Section 4.1.1), and required iterative interaction with the academic supervisor throughout the two-term project timeline (Figure I.2).
A3	Innovation	Introduces a novel combination of road-class access constraints, PCU-based load, capacity-aware cost, and pheromone discouragement within a single ACO framework (Section 3.9), going beyond directly reviewed prior methods (Section 2.7).
A4	Consequences to society and the environment	A successfully deployed version of this framework could reduce congestion, travel delay, fuel consumption, and emissions in Dhaka (Sections 5.7 and 5.8), while a poorly calibrated or unfairly configured priority scheme could disadvantage some road users, which is why fairness and transparency are explicitly discussed in Section 5.3.

continued on next page

Table VI.2 continued from previous page

Attribute	Description	Relevance to this thesis
A5	Familiarity	Combines familiar undergraduate-level techniques (graph algorithms, basic statistics) with less familiar, thesis-specific techniques developed for this project (the priority-discouragement mechanism and the multi-factor congestion-aware cost function), requiring the author to extend beyond standard course-work material.
A6	Leadership and management	As a single-author thesis, the author was solely responsible for project planning (Section 1.9), time management across the Gantt-chart schedule, and prioritization of tasks across dataset preparation, implementation, and experimentation phases, under the supervisor’s periodic review.
A7	Consequences of failure	An error in the cost model, access matrix, or routing logic could produce misleading conclusions about which method performs better, or, in a real deployment, could recommend infeasible or unsafe routes; this risk is mitigated in this thesis through the independent manual path-evaluation case study (Section 4.6.3), which cross-checks the automated ACO output against hand-verified alternative routes under the same cost model, and through the common-layer verification protocol (Section 4.4), which proves by byte-level comparison that the comparison instrumentation altered no algorithm’s native results.

6.4 Mapping of the Proposed Work with Complex Engineering Attributes

Table VI.3 connects each major system component from Chapters III–IV to the CEP/CEA attributes it most directly engages.

6.5 Summary

The thesis engages substantively with all seven CEP attributes, most centrally P2 (conflicting individual-versus-network-wide optimality) and P7 (interdependence of the traffic-flow, cost, and routing sub-models), and all seven CEA attributes, most centrally A3 (innovation in combining access, load, capacity, and discouragement) and A7 (managing modeling-error risk through independent manual verification). Despite being a focused, single-author undergraduate thesis, the project therefore exercises the depth of knowledge, analytical rigor, and engineering judgment characteristic of a complex engineering problem rather than a routine application of a single established algorithm.

Table VI.3: Consolidated mapping of major system components to CEP/CEA attributes.

System component	Primary CEP/CEA attributes	Justification
Dhaka road-network dataset preparation (Section 3.3)	P1, P4, A1	Requires domain knowledge of road hierarchy and careful use of open mapping resources to build an accurate, unfamiliar-scale dataset.
Congestion-aware cost model (Section 3.5)	P1, P3, P7, A3	Requires deep, interdependent analysis combining several transportation and graph-theoretic sub-models into one innovative cost function.
Independent ACO (Section 3.6.1)	P1, P4	Applies established metaheuristic theory as a controlled baseline.
Priority-Discouragement ACO (Section 3.6.2)	P2, P7, A3, A7	Directly manages the conflicting-requirements trade-off between individual and network-wide optimality through an innovative, interdependent mechanism whose correctness must be carefully verified.
Manual path-evaluation module (Section 3.7.2)	A7	Mitigates the consequences of a possible modeling or implementation error by providing an independent, human-interpretable verification path.
Result comparison and metrics (Section 4.7)	P3, P6	Requires statistically and practically meaningful interpretation of results relevant to transportation-authority stakeholders.

CHAPTER VII

Conclusion and Future Work

7.1 Summary of the Study

This thesis developed and evaluated a congestion-aware, priority-ordered Ant Colony Optimization framework for network-level route optimization on central Dhaka. Motivated by Dhaka's severe congestion and the limits of congestion-blind shortest-path routing (Chapter I), it reviewed related work and re-implemented five related methods on the thesis's own dataset to empirically confirm their reported behavior (Chapter II). A complete methodology followed (Chapter III): a 1,168-node, 3,473-link Dhaka road-network dataset; a stochastic, road-class-calibrated traffic-flow model; a multi-factor cost model combining distance, capacity, PCU-weighted load, congestion, and access permission; and two comparable ACO strategies – Independent ACO (baseline) and Priority-Discouragement ACO, which orders queries by importance and discourages pheromone on already-used links while propagating congestion between queries. Both were implemented in a modular C++17 application and evaluated on the same 500-query dataset under identical parameters, with a common evaluation layer – verified byte-identical to native outputs – placing eleven method rows on identical metrics (Chapter IV). The broader engineering, ethical, and societal implications were then examined (Chapters V–VI).

Concretely, the study delivered: (1) a cleaned 1,168-node, 3,473-link Dhaka dataset with matching vehicle, access-matrix, traffic-flow, and query data; (2) a congestion-aware road-cost model usable as an ACO heuristic; (3) a novel Priority-Discouragement ACO strategy evaluated against an Independent ACO baseline under matched conditions; (4) a comparative evaluation supported by a manual path-evaluation case study; and (5) a reusable common evaluation layer enabling fair, same-dataset comparison of heterogeneous methods without modifying any algorithm.

7.2 Major Findings

Relative to the Independent ACO baseline, Priority-Discouragement ACO:

- reduces total route cost from 3.764944×10^8 to 3.454061×10^8 ($\approx 8.26\%$);
- reduces average network congestion from 2.174554 to 1.603040 ($\approx 26.28\%$);

- improves load balance from 0.395797 to 0.443135 ($\approx 11.96\%$, CV-based) and from 0.300 to 0.388 ($\approx 29.1\%$, Jain’s index); and
- completes all 500 jointly optimized queries in 13.3 s (≈ 27 ms/query), $2.04\times$ the baseline’s 6.5 s.

Since both methods share an identical cost model and ACO parameters, these differences are attributable specifically to priority ordering and pheromone discouragement, and are larger on the complete network than on the earlier sparse extract. The common evaluation layer added two further findings: both methods route exactly the same 351 of 500 queries (the 149 failures are structurally infeasible, not starved by the mechanism), with Method 2 consolidating each query’s fleet onto far fewer paths (1.06 vs. 1.75 paths/query); and in the eleven-row comparison, the proposed method is the only one that jointly optimizes the whole network for hundreds of simultaneous vehicles, versus one vehicle (Cheng [4]), 50 independent pairs (Wu [3]), one accident with negligible network-wide change (Zhang [5]), no routing (signal-timing), or bicycle-lane design rather than vehicle routing (Physarum [12]). A manual case study further confirmed the automated route outperformed two hand-built alternatives by 42.6% and 7.2% in cost (Section 4.6.3).

7.3 Limitations of the Study

- **Infeasible queries.** 149 of 500 queries (29.8%) are structurally infeasible on the cleaned dataset (disconnected under access constraints, several via out-degree-zero source nodes, Section 3.4) and fail identically under both methods; this does not bias the comparison but means results describe a smaller effective demand than the nominal 500.
- **Synthetic traffic data.** Flows are Poisson-simulated (Section 3.4) rather than measured, since live sensor/GPS data was unavailable; absolute accuracy against Dhaka’s real traffic state is not independently validated.
- **Restricted study area.** The dataset covers central Dhaka ($\approx 17 \times 28$ km) rather than the full metropolitan network, limiting generalization to peripheral areas.
- **Static evaluation snapshot.** Both methods are evaluated at one simulated time step, not across a full day or peak/off-peak scenarios.
- **Fixed parameters.** ACO (20 ants, 30 iterations, $\alpha = 1$, $\beta = 2$, $\rho = 0.20$) and priority-discouragement ($\theta = 0.08$, $w_p = 0.65$, $w_d = 0.35$) parameters were fixed rather than systematically tuned (e.g., via PSO [10] as in Section 2.6), and the three

coordination components were evaluated jointly rather than ablated individually (Section 4.6.4).

- **Limited cross-method comparability.** Cost/congestion units differ per paper, so cross-project comparison is restricted to capabilities, success rates, runtimes, and fairness indices (Section 3.8).
- **No field validation.** The framework has been evaluated only in offline simulation, not integrated into a live system or tested with real drivers.
- **Simplified safety factors.** The cost model omits road-surface condition, lighting, pedestrian density, and accident history (Section 5.5).

7.4 Future Recommendation

- (1) **Real-time or historical traffic data** to replace or calibrate the synthetic flow model (Section 3.4).
- (2) **Repairing dataset connectivity and demand generation** by auditing disconnected components and constraining query generation to reachable origin–destination pairs.
- (3) **Expansion to the full Dhaka metropolitan network**, benefiting from the parallelization and hierarchical-routing strategies of Section 5.1.
- (4) **Time-varying, multi-period evaluation** across a full day or peak/off-peak scenarios.
- (5) **Component-wise ablation** of priority ordering, discouragement, and traffic propagation individually.
- (6) **Automated parameter tuning** (e.g., PSO or grid/Bayesian search over θ, w_p, w_d).
- (7) **Safety- and comfort-aware cost extensions** incorporating road condition, accident history, or pedestrian density into Equation III.7.
- (8) **Emergency and priority-vehicle case studies**, assigning ambulances/fire vehicles maximal priority under Equation III.12.
- (9) **Growing the common comparison suite:** any project writing the agreed `common_result.csv` schema is picked up automatically (Section 4.3).
- (10) **Practical navigation-system integration** packaging the cost model and routing engine as a queryable service.

7.5 Closing Statement

This thesis demonstrates that treating a megacity’s road network as one coordinated system, rather than a collection of independent trips, yields measurably better traffic outcomes, and provides a working, verifiable framework realizing this idea for Dhaka. By integrating vehicle-class access, PCU-based load, capacity-aware cost, and priority-aware pheromone discouragement within a single ACO framework – shown in Chapter II to be absent from directly related prior work and verified through re-implementation of three closely related methods on the same dataset – the thesis addresses a genuine, verifiable gap. Priority-Discouragement ACO outperformed its matched baseline on every metric, served the same demand while consolidating it onto fewer, more balanced routes, within real-time-capable computation; and the common evaluation layer contributed a reusable standard of evidence for comparing heterogeneous optimization methods. While synthetic traffic data, a bounded study area, and the lack of field validation leave clear work ahead, the consistent, verified improvements provide a solid foundation for future congestion-aware navigation and priority-routing systems for Dhaka and similarly congested megacities.

REFERENCES

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] M. Dorigo, V. Maniezzo, and A. Coloni, “Ant system: optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [3] C. Wu, S. Zhou, and L. Xiao, “Dynamic path planning based on improved ant colony algorithm in traffic congestion,” *IEEE Access*, vol. 8, pp. 180773–180783, 2020.
- [4] J. Cheng, “Dynamic path optimization based on improved ant colony algorithm,” *Journal of Advanced Transportation*, vol. 2023, 2023.
- [5] H. Zhang, S. Guo, X. Long, and Y. Hao, “Combined dynamic route guidance and signal timing optimization for urban traffic congestion caused by accidents,” *Journal of Advanced Transportation*, vol. 2023, 2023.
- [6] R. Bader, J. Dees, R. Geisberger, and P. Sanders, “Alternative route graphs in road networks,” in *Theory and Practice of Algorithms in (Computer) Systems (TAPAS 2011)*, vol. 6595 of *Lecture Notes in Computer Science*, (Berlin, Germany), pp. 21–32, Springer, 2011.
- [7] U.S. Bureau of Public Roads, *Traffic Assignment Manual*. Washington, DC, USA: U.S. Department of Commerce, 1964.
- [8] R. Jain, D.-M. Chiu, and W. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” Tech. Rep. DEC-TR-301, Digital Equipment Corporation, 1984.
- [9] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004.
- [10] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of the IEEE International Conference on Neural Networks*, pp. 1942–1948, 1995.
- [11] M. Dorigo and L. M. Gambardella, “Ant colony system: a cooperative learning approach to the traveling salesman problem,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.

- [12] M. A. H. Akhand, M. A. Habib, M. A. S. Kamal, and N. Siddique, “Physarum-inspired bicycle lane network design in a congested megacity,” *Applied Sciences*, vol. 11, no. 15, 2021.
- [13] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA, USA: MIT Press, 3rd ed., 2009.
- [14] T. L. Saaty, “A scaling method for priorities in hierarchical structures,” *Journal of Mathematical Psychology*, vol. 15, no. 3, pp. 234–281, 1977.
- [15] T. Stützle and H. H. Hoos, “MAX-MIN ant system,” *Future Generation Computer Systems*, vol. 16, no. 8, pp. 889–914, 2000.
- [16] H. Yu, “The impact of path planning model based on improved ant colony optimization algorithm on green traffic management,” *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 6, pp. 1065–1074, 2024.
- [17] Y. Wei, N. Jiang, Z. Li, D. Zheng, M. Chen, and M. Zhang, “An improved ant colony algorithm for urban bus network optimization based on existing bus routes,” *ISPRS International Journal of Geo-Information*, vol. 11, no. 5, 2022.
- [18] S. W. Lee, K. S. Heo, M. A. Kim, D. K. Kim, and H. Choi, “Multiple-junction-based traffic-aware routing protocol using aco algorithm in urban vehicular networks,” *Sensors*, vol. 24, no. 9, 2024.
- [19] E. Ghaffari, A. M. Rahmani, M. Saberikamarposhti, and A. Sahafi, “An optimal path-finding algorithm in smart cities by considering traffic congestion and air pollution,” *IEEE Access*, vol. 10, pp. 55126–55135, 2022.
- [20] W. Zhu, W. Cai, and H. Kong, “Optimal path planning based on ACO in intelligent transportation,” *International Journal of Cognitive Computing in Engineering*, vol. 6, pp. 441–450, 2025.
- [21] H. Wang, “Dynamic path optimization of cold chain logistics based on improved ant colony algorithm from a low-carbon perspective,” *Sustainable Operations and Computers*, vol. 6, pp. 297–305, 2025.
- [22] M. Korzeń and M. Kruszyna, “Public transport planning using modified ant colony optimization,” *Sustainability*, vol. 17, no. 6, 2025.
- [23] S. Liu and H. Wu, “Analysis of the application of path finding system based on efficiency improvement in smart tourism,” *Intelligent Systems with Applications*, vol. 20, 2023.

- [24] T. Arpitha, D. Chouhan, and J. Shreyas, “An efficient ACO-inspired multi-path routing for source location privacy with dynamic phantom node selection scheme in IoT environments,” *Soft Computing*, vol. 28, no. 23, pp. 13149–13166, 2024.
- [25] A. S. B. Shahadat, M. A. H. Akhand, and M. A. S. Kamal, “Visibility adaptation in ant colony optimization for solving traveling salesman problem,” *Mathematics*, vol. 10, no. 14, 2022.
- [26] M. A. H. Akhand, S. I. Ayon, S. A. Shahriyar, N. Siddique, and H. Adeli, “Discrete spider monkey optimization for travelling salesman problem,” *Applied Soft Computing*, vol. 86, 2020.
- [27] J. G. Wardrop, “Some theoretical aspects of road traffic research,” *Proceedings of the Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952.
- [28] Y. Sheffi, *Urban Transportation Networks: Equilibrium Analysis with Mathematical Programming Methods*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1985.
- [29] S. M. Labib, H. Mohiuddin, I. Mohammad, A. Hasib, S. H. Sabuj, and S. Hira, “Integrating data mining and microsimulation modelling to reduce traffic congestion: A case study of signalized intersections in dhaka, bangladesh,” *Urban Science*, vol. 3, no. 2, p. 41, 2019.
- [30] G. Musolino, A. Polimeni, C. Rindone, and A. Vitetta, “Travel time forecasting and dynamic routes design for emergency vehicles,” *Procedia - Social and Behavioral Sciences*, vol. 87, pp. 193–202, 2013.
- [31] L. Yan, P. Wang, J. Yang, Y. Hu, Y. Han, and J. Yao, “Refined path planning for emergency rescue vehicles on congested urban arterial roads via reinforcement learning approach,” *Journal of Advanced Transportation*, vol. 2021, 2021.
- [32] Z. Hao, Y. Wang, and X. Yang, “Every second counts: A comprehensive review of route optimization and priority control for urban emergency vehicles,” *Sustainability*, vol. 16, no. 7, 2024.
- [33] Y. Liu, L. Huo, J. Wu, and A. K. Bashir, “Swarm learning-based dynamic optimal management for traffic congestion in 6G-driven intelligent transportation system,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 7, pp. 7831–7846, 2023.
- [34] S. Lämmer and D. Helbing, “Self-control of traffic lights and vehicle flows in urban road networks,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 4, 2008.

- [35] M. A. H Akhand, P. C. Shill, M. S. Sabuj, Z. Afrin, and M. M. Hafizur Rahman, "A PSO based transportation network design optimization of the mega city dhaka," in *Proc. Int. Conf. Electrical Engineering and Information & Communication Technology (iCEEICT)*, (Dhaka, Bangladesh), 2015.
- [36] K. A. Ahad, D. Yasmeen, and R. R. Khan, "Real time traffic congestion analyzer of dhaka city," *IST Journal on Business & Technology*, vol. 8, no. 1, 2017.
- [37] S. M. Sabbir, "Traffic volume in dhaka city at selected priority junctions," *Journal of Transportation Engineering and Traffic Management*, vol. 3, no. 1, pp. 1–10, 2022.
- [38] Hasinae-Jannat, "Pedestrian road crossing behavior at signalised intersections and mid block sections in dhaka city," Master's thesis, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh, 2013.
- [39] M. S. Hossain, A. Anjum, M. A. H. Akhand, and N. Siddique, "Suitable base station selection for post disaster quick road network recovery in a vulnerable urban area," in *Proc. 2020 IEEE Region 10 Symposium (TENSYP)*, (Dhaka, Bangladesh), 2020.
- [40] F. Sultana, S. M. Rahman, M. A. H. Akhand, S. I. Hossain, and M. A. S. Kamal, "Route optimization for emergency vehicles: Analyzing spatial and temporal dependencies using graph convolutional network." Unpublished manuscript, Dept. of Computer Science and Engineering, Khulna University of Engineering & Technology, Khulna, Bangladesh.
- [41] Transportation Research Board, *Highway Capacity Manual*. Washington, DC, USA: National Academies Press, 6th ed., 2016.

APPENDIX A

Representative Source Code Excerpts

This appendix reproduces short, representative excerpts from the C++17 implementation, corresponding to the formulas and modules of Chapters III and IV; the full source tree additionally includes dataset-generation, I/O, and utility modules described narratively in Section 4.1.1.

A.1 Ant Path Construction (IndependentACO.cpp)

The core of the transition rule (Equation III.10), shared by both routing methods:

```
1 for (int edgeId : dataset.adjacency[current]) {
2     double cost = CostModel::vehicleRoadCost(dataset, edgeId, query.
        vehicleType, traffic);
3     if (!std::isfinite(cost)) continue;
4     double tau = std::max(1e-9, pheromone[edgeId]);
5     double eta = 1.0 / (cost + 1e-9);
6     double weight = std::pow(tau, alpha) * std::pow(eta, beta); // P_e
        ^k numerator
7     candidates.push_back({ edgeId, weight });
8     totalWeight += weight;
9 }
10 // roulette-wheel selection over 'candidates' weighted by 'weight /
    totalWeight'
```

Listing A.1: Core of the ACO transition rule implementing Equation III.10.

A.2 Traffic Flow Generation (FlowGenerator.cpp)

The Poisson-based inflow/outflow step implementing Equation III.3 and the occupancy/PCU-load update:

```
1 double lambda = baseLambda(roads[e].type) * vehicleWeight[q];
2 std::poisson_distribution<int> inflowDist(std::max(0.1, lambda));
3 std::poisson_distribution<int> outflowDist(std::max(0.1, lambda *
    0.85));
4 int inflow = inflowDist(rng);
5 int outflow = std::min(current[e][q] + inflow, outflowDist(rng));
6 current[e][q] = std::max(0, current[e][q] + inflow - outflow);
7 double pcuLoad = current[e][q] * vehicles[q].pcu;
```

Listing A.2: Core of the Poisson flow generation implementing Equation III.3.

A.3 Cost Model Module (CostModel.cpp)

The core road-cost function implementing the Congestion-Aware Road Cost Rule of Section 3.5:

```
1 double CostModel::vehicleRoadCost(const Dataset& dataset, int edgeId,
2   int vehicleType, const std::vector<std::vector<double>>& traffic)
3   {
4     const Road& road = dataset.roads[edgeId];
5     double access = dataset.accessMatrix[road.type][vehicleType];
6     if (access <= 0.0) return INF_COST;
7     double congestion = congestionCoefficient(dataset, edgeId, traffic
8       ); //  $C_e$ 
9     return road.length * (1.0 + congestion) / access; //  $cost_e(q)$ 
```

Listing A.3: Core road-cost function implementing the Rule of Section 3.5.

A.4 Priority Score and Discouragement Mechanism (PriorityDiscouragementACO.cpp)

The importance score and discouragement update implementing the Rule of Section 3.6.2 (code variables gamma, delta, theta correspond to the thesis symbols w_p , w_d , θ):

```
1 double PriorityDiscouragementACO::queryImportance(const Query& query)
2   {
3     double distance = shortestBaseDistance(query.source, query.
4       destination);
5     if (!std::isfinite(distance)) distance = 1e9;
6     return gamma * query.priority + delta * (1.0 / (distance + 1e-9));
7     //  $I(q)$ 
8   }
9 void PriorityDiscouragementACO::discourageUsedEdges(std::vector<
10   double>& pheromone,
11   const Query& query, const std::vector<RouteAssignment>&
12   assignments) {
13   for (const auto& a : assignments)
14     for (int edgeId : a.edges)
15       usage[edgeId] += (double)a.vehicleCount / std::max(1, query.
16         vehicleCount); //  $\phi_e$ 
17   for (size_t e = 0; e < pheromone.size(); ++e)
18     pheromone[e] = std::max(1e-9, pheromone[e] * (1.0 - theta * std.
19       ::min(1.0, usage[e])));
```

13 }

Listing A.4: Query-importance scoring and pheromone discouragement implementing the Rule of Section 3.6.2.

A.5 Metrics Calculation Module (**Metrics.cpp**)

The core of the total-cost, average-congestion, and load-balance computation (Equations III.15–III.17), used to produce Table IV.11:

```

1  for (const auto& a : result.assignments) {
2      double cost = CostModel::pathCost(dataset, a.edges, a.vehicleType,
          traffic);
3      if (std::isfinite(cost)) metric.totalCost += cost * a.vehicleCount
          ; // TC
4  }
5  for (size_t e = 0; e < dataset.roads.size(); ++e)
6      congestionSum += CostModel::congestionCoefficient(dataset, (int)e,
          traffic);
7  metric.averageCongestion = congestionSum / dataset.roads.size(); //
    AC
8  // utilization[e] = load[e] / capacity[e]; loadBalance = 1 / (1 +
    stddev(utilization)/mean(utilization))

```

Listing A.5: Core metrics computation implementing Equations III.15–III.17.

A.6 Common Evaluation Layer Output

The common evaluation layer (Sections 3.7.3, 4.3) writes one output/common_result.csv per project using a fixed 18-column schema; the root-level aggregator merges all projects into final_common_comparison.csv. Table A.1 reproduces the directly comparable columns of all eleven rows.

A.7 Sample Dataset Records

```

1  # dhk_links_clean.csv (excerpt)
2  lane_count,edge_id,from,to,length,road_type
3  3,1002,1,2,450.0,arterial
4
5  # vehicles.csv (full)
6  vehicle_id,name,pcu,length
7  0,motorcycle,0.3,2
8  1,cng,0.7,3
9  2,private_car,1,4.5
10 3,bus,2.5,10

```

Table A.1: Aggregated common-layer comparison (`final_common_comparison.csv`, complete-dataset run of 4 July 2026). “L.B.” is the Jain fairness index; N/A entries are honest by-design gaps documented in each row’s notes field.

method_name	nodes	links	succ.	rate	runtime	L.B.
BasePaper_ImprovedACO_PSO	1168	3477	34/50	0.680	6903.0	0.709
DhakaSignal_AutoOptimizedPlan	1168	3564	N/A	N/A	20254.4	N/A
Proposed_IndependentACO	1168	3473	351/500	0.702	6517.2	0.300
Proposed_PriorityDiscouragementACO	1168	3473	351/500	0.702	13310.6	0.388
Cheng2023_ImprovedACO_base	1061	3084	1/1	1.000	42.1	N/A
Cheng2023_ImprovedACO_peak_eq9	1061	3084	1/1	1.000	68.7	N/A
Zhang2023_A_Original_NoOptimization	1168	7124	58/60	0.967	620.3	0.543
Zhang2023_B_DynamicRouteGuidance	1168	7124	59/60	0.983	620.3	0.543
Zhang2023_C_SignalTimingOptimization	1168	7124	58/60	0.967	620.3	0.543
Zhang2023_D_Combined_RouteGuidance_SignalOpt	1168	7124	59/60	0.983	620.3	0.543
PhysarumInspired_BLN_Dhaka	1098	3366	120/120	1.000	7.3	0.181

```

11 4,rickshaw_bicycle,0.5,2.5
12
13 # output/metrics.csv (full, primary experimental result)
14 method,total_cost,average_congestion,load_balance
15 independent_aco,376494410.904167,2.174554,0.395797
16 priority_discouragement_aco,345406051.470833,1.603040,0.443135

```

Listing A.6: Excerpt from `dhk_links_clean.csv` and full contents of `vehicles.csv` and `output/metrics.csv`.

A.8 Reproducing the Experiments

Consistent with the responsible-research commitment of Section 5.3:

- (1) **Prerequisites.** A C++17-capable compiler (Visual Studio, Section 4.1) and Python 3 for the common-layer aggregator; no external libraries required.
- (2) **Project layout.** The modules of Section 4.1.1 compile into one executable (`route_optimizer`), reading a `data/` directory of input CSVs (Section 3.3) and writing to `output/`.
- (3) **Configuration.** The checked-in `data/config.csv` already holds the parameter values of Table III.8, with `regenerate_dataset=0` and `use_dhaka_network=1`, so existing CSVs are loaded as-is.
- (4) **Build and run.** Building in Release (x64) and running the binary executes the full pipeline (Section 3.1): load dataset, run Method 1, run Method 2, compute metrics, write the outputs of Table IV.5.

- (5) **Determinism.** With `random_seed=42` fixed and seeding every stochastic component, re-running reproduces Table IV.11 exactly, modulo platform-specific floating-point/PRNG differences between compilers.
- (6) **Manual evaluation.** Placing `manual_node_paths.csv` (`method, query_id, node_path`, Section 3.7.2) in `output/` before running triggers the manual evaluation step, producing `manual_evaluated_paths.csv` and `manual_metrics.csv`.
- (7) **Cross-method aggregation.** After running each comparison project, `python aggregate_common_results.py` regenerates `final_common_comparison.csv`, reproducing Table IV.12.